

Correct-and-then-Detect: Adaptive Fault Detection through Data-driven Digital Twins and Transfer Learning

ARTICLE INFO

Keywords:

Data-driven digital twin,
predictive maintenance,
fault detection,
domain adaptation,
transfer learning,
robot arm

ABSTRACT

In a digital twin-based fault detection framework, it is essential to have a reliable and robust digital twin to generate reference values representing nominal behaviors. Traditional digital twin modelling approaches require deep understanding of functioning principles and failure mechanisms of the system, which are not always available in practice. Furthermore, as systems operate, maintaining an accurate digital twin becomes challenging as system's behavior tends to deviate from its nominal cases due to various factors like aging, uncertainty and sim-to-real gaps. To address these issues and better support predictive maintenance, this paper proposes an integrated framework for data-driven digital twin modeling and fault detection. First, we develop a design-of-experiment-based approach to generate training data that can efficiently cover the potential task space. The generated training data are then used to train a data-driven digital twin, *i.e.*, a deep learning model that predicts system performances at a given time instant given observed covariates from a time window prior to it. Next, we develop a new adaptive fault detection method based on the developed data-driven digital twin. The algorithm works in an adaptive manner: first, the reference values generated by the digital twin is screened and corrected to remove potential contaminations from faulty signals in the context window. Then, the corrected reference values are used in a residual-based framework for fault detection. Finally, a transfer learning mechanism is applied in the developed framework to make sure the performance of the data-driven digital twin and fault detection methods remain robust under system evolution and cross-domain distribution shifts. Real experiments are conducted on two real robots. The results confirm that the data-driven digital twin model accurately captures the actual behavior of the physical entity and that the developed adaptive fault detection methods provide more accurate and robust results compared to traditional methods.

1. Introduction

Complex industrial systems like industrial robots typically operate under complex and strongly nonlinear environments. Their operational behavior evolves over time because of aging, parameter drift, and environmental variations [1, 2]. Reliable condition-monitoring and fault detection for such systems are therefore important but challenging issues [3]. Traditionally, fault detection methods for complex industrial systems mainly rely on statistical analyses of measured data and residuals [3]. Typical methods in this category include signal processing-based approaches, rule-based systems, and model-based methods [3, 4]. A detailed review of such methods is presented in Sect. 2.1. As can be seen in Sect. 2.1, a major limitation of these traditional methods is that they require handcrafted features or accurate mathematical models, which are often unavailable for complex systems operating under frequently-changing operating conditions.

As an alternative to the statistical analyses of measured data and residuals, data-driven fault detection methods become increasingly common as they do not require handcrafted features or explicit system models [3, 4]. Instead, they learn fault-discrimination patterns directly from historical data using machine learning or deep learning models [3, 4] (see Sect. 2.1 for a detailed review). As shown in Sect. 2.1, while performing generally well on high-dimensional nonlinear systems, the machine learning/deep learning-based methods generally require large amounts of labeled fault data for training. In industrial systems, fault events are often rare and uncontrollable, which limits practical deployment.

To reduce dependence on labeled fault data, digital twin technology has been introduced into fault detection [5, 6]. In these methods, a digital twin is developed to represent nominal system behavior and supplies reference values for measured data. Faults are then detected from residuals between measurements and their references. As discussed in Sect. 2.3, existing digital twin faces a critical challenge: developing high-fidelity physics-based or simulation-based digital twins relies on in-depth domain knowledge and precise parameter calibration, making them difficult to build, especially for complex systems under long-term operation [7, 8].

In recent years, data-driven digital twins have attracted increasing attention as an alternative paradigm that integrates data-driven modeling with virtual–physical mapping [9, 10, 11, 12]. The core idea is to learn the nominal

behavior of the system from healthy operational data and to formulate fault detection as the identification of deviations from nominal behavior [13]. Compared with direct fault classification approaches, this paradigm offers clear advantages in terms of data availability and engineering feasibility [14]. **The nominal predictor is not itself the fault detector: it supplies an operating-condition-dependent reference from which a residual is constructed, and the residual is subsequently used to support fault detection.** However, as analyzed in detail in Sect. 2.3, existing data-driven digital twin methods still face three challenges: (1) most existing data-driven digital twin models lack systematic experimental design to ensure sufficient coverage of the operational space in the training phase, which limits their accuracy and generalization capability; (2) the developed digital twins easily become outdated when system behavior changes during operation; (3) the reference values predicted by the digital twins can be contaminated by fault-affected inputs because most data-driven digital twins use a rolling window to make predictions.

To address these limitations, this paper develops a digital-twin-driven fault detection framework that is able to (1) efficiently design training data generation process to ensure the coverage of the generated training data; (2) remains robust under system evolution and cross-domain distribution shifts ; and (3) remains insensitive to the data contamination brought by faulty signals. The main contributions are:

- A DoE-guided data acquisition and rolling-horizon modeling procedure is established. Through Halton sampling, the generated training data fully cover a bounded task-configuration envelope, making sure the trained model fully learns the typical application scenarios of the target system;
- A transfer learning-based framework is developed to update the developed data-driven digital twins under system evolution and cross-domain distribution shifts.
- A threshold-triggered contamination-suppression mechanism, named Prediction-based Recursive Observation Feedback (PROF), is introduced to avoid data contamination caused by including faulty signals into the context window of the data-driven digital twin.
- An open dataset from two real robots is released to validate the developed data-driven digital twin modeling and fault detection framework.

It should be noted that the main contribution here is the integrated data-driven fault detection framework, including nominal behavior modeling, transfer-learning-based digital twin updating, and PROF-enhanced residual fault detection, rather than the performance of its individual building blocks like prediction accuracy of the digital twin.

The remainder of this paper is organized as follows. Section 2 reviews related work on fault detection and digital twin modeling. Section 3 presents the proposed data-driven digital twin framework with transfer learning. Section 4 introduces the residual-based fault detection method and the PROF mechanism. Section 5 describes the experimental setup. Section 6 reports and analyzes the experimental results. Section 7 concludes the paper.

2. Related work

In this section, we review the related works with a focus on their limitations. In Sect. 2.1, we review traditional approaches for **fault detection**. Recent works on digital twin-based fault detection are discussed in Sect. 2.2. In Sect. 2.3, **we** discuss data-driven digital twin modeling.

2.1. Traditional fault detection methods

According to [4, 15], traditional fault detection methods can be classified into model-based, signal-based, and data-driven approaches. Model-based techniques use explicit mathematical representations of nominal system dynamics to detect deviations between observed and predicted behavior [16, 17]. For example, observer-based residual generation methods construct state estimators such as Kalman filters and Luenberger observers to compare measured outputs with model predictions, where residual signals are monitored for significant deviations indicating faults. Prior work has applied such observers for actuator fault diagnosis in benchmark systems, demonstrating their ability to detect anomalies by thresholding estimation residuals [18]. Fuzzy model-based observers have also been designed for detecting sensor faults in chemical process systems, where multiple observer structures are evaluated based on their residual generation performance [19]. While effective when accurate models exist, they struggle in complex systems due to the difficulty of capturing nonlinear interactions and detailed normal/fault behaviors, especially for components like actuators and sensors [20].

Signal-based methods extract features from raw measurements, such as spectral or statistical characteristics, and compare them to reference profiles [21, 22]. For example, time–frequency spectral amplitude modulation methods extract characteristic fault features from vibration signals by combining short-time Fourier transform with weighted spectral amplitude analysis, enabling effective detection of bearing faults under noisy conditions [23]. Similarly, advanced machine health monitoring approaches leverage time–frequency feature extraction techniques to identify modulation patterns induced by transmission element faults, such as amplitude and frequency modulations in rotating machinery vibration signals [24]. These approaches avoid reliance on explicit system models but depend on carefully designed feature extraction and thresholding, which are difficult to generalize across varying operating conditions or high-dimensional industrial data.

Data-driven methods leverage machine learning to infer fault patterns directly from data without explicit physical models. They can automatically capture complex temporal and spatial correlations using approaches such as support vector machines or neural networks [25, 26, 27, 28]. For example, data-driven fault diagnosis methods have been widely applied in rotating machinery, where convolutional neural networks are trained to classify fault types directly from raw vibration signals without explicit feature design [29]. Deep learning models have also been used to detect and distinguish incipient and compound faults in industrial systems by learning complex representations from multi-sensor time-series data, often outperforming traditional machine learning classifiers on benchmark datasets [30]. These approaches avoid reliance on explicit physical models and can automatically capture intricate temporal and spatial correlations in the data. However, most data-driven methods require large quantities of labeled fault data for supervised training. In real industrial environments, fault events are often rare to observe. Obtaining enough labeled failure data is often difficult considering the constraints of costs and safety [30, 31, 32].

2.2. Digital twin-based fault detection

Digital twin technology provides a virtual replica of a physical system that can be used for monitoring, simulation, and fault diagnosis [33, 14]. In fault detection applications, digital twins are often constructed using physics-based or simulation models that emulate the detailed behavior of the physical entity. These high-fidelity models can generate synthetic data under various operating conditions, support predictive analysis, and enable the exploration of fault scenarios that are difficult to observe directly on the physical system [34, 35]. For example, digital twin models of electric machines have been used to simulate motor dynamics and extract fault-relevant features for diagnostic tasks [36]. Similarly, physics-based digital twin models have been applied to gearbox fault diagnosis, where multi-physical simulations generate virtual vibration data for training diagnostic models [37]. Recent work has attempted to integrate virtual and physical data for fault detection and diagnosis. Through physical-virtual data fusion or hybrid modeling, the quality of synthetic fault data and diagnosis performance can be improved [37, 38].

While physics/simulation-based digital twins offer substantial potential for fault detection, they also face significant challenges. Constructing detailed physics models that faithfully represent all relevant behaviors of complex systems is resource intensive, requiring comprehensive domain knowledge and extensive calibration. In addition, such models often depend on simplified assumptions or idealized conditions that limit their ability to capture real-world variability in operational environments, sensor noise, and degradation processes. These limitations make it difficult to scale physics-based twin models to diverse systems or to maintain them over long operating life cycles [35, 14]. Moreover, discrepancies between simulated and real system behavior can degrade diagnostic accuracy when models are applied in practice.

2.3. Data-driven digital twin modeling

As an alternative to physics-based digital-twin modeling, data-driven approaches have been used increasingly. These approaches train statistical or machine-learning models on sensor data collected under normal conditions, without requiring detailed physical or simulation models [39, 40, 41]. Data-driven twins capture system behavior directly from data streams and have been applied to real-time monitoring, anomaly detection, and fault diagnosis. For example, streaming sensor data have been used to update machine-monitoring models and identify deviations indicative of faults [42]. Long short-term memory (LSTM) networks have also been embedded in digital twins for fault diagnosis of electro-hydraulic actuators, demonstrating that learned representations can support detection without explicit physical models [43].

Despite these advances, current data-driven digital twin research exhibits notable limitations as well. First, most existing studies collect training data from available operational traces without systematic design of experiments (DoE) to ensure coverage of relevant system behaviors across different operating conditions. Without proper experiment

designs, the learned models may lack sufficient diversity to generalize beyond the scenarios represented in the training set. Second, although some works integrate machine learning within digital twin structures, few explicitly address transfer learning or domain adaptation to enhance model generalization when the operating domain shifts or when the digital twin is deployed on new systems or environments. As a result, data-driven twins trained on one dataset may perform poorly when applied to unseen conditions.

3. Data-driven digital twin modeling with design-of-experiment and transfer learning

In this section, we develop a data-driven digital twin framework for nominal behavior modeling. Section 3.1 formalizes the problem setting. Section 3.2 constructs the rolling-horizon predictive digital twin through design-of-experiments–based data acquisition, temporal sample construction, and sequence model learning. Section 3.3 presents a transfer learning strategy for adaptive updating under system evolution.

3.1. Problem definitions

We consider a physical system that generates a multivariate response that is observable at time instants $t_1, t_2, \dots, t_n$. The objective of a data-driven digital twin is to predict the system response by mapping historical observations to the future evolution of a quantity of interest over a finite prediction horizon. To formally define the problem, let us assume that we observe the system's response $\mathcal{Y}$ and covariate $\mathcal{X}$ at fixed instants. Let w denote the size of a sliding window, inside which the covariates are used to build the data-driven digital twin. Conceptually, our task is to build a predictive operator mapping from the observation history to the predicted performance at a future instant. Let $\mathcal{X}^*$ denote the space of finite observation histories and let $\mathcal{Y}^H$ denote the space of target trajectories over a horizon of length H . At each time step t , the prediction task is described by a distribution-conditioned operator

$$\mathcal{T}_{\mathbb{P}_t} : \mathcal{X}^* \rightarrow \mathcal{Y}^H, \quad (1)$$

where $\mathbb{P}_t$ denotes the data distribution governing the system observations at time t . As the system operates, the data-driven digital twin in (1) can be applied sequentially to predict the system performance online. When new data become available, the context window w moves forward in a rolling-horizon manner to reflect system status changes. In Sect. 3.2, we first present a new DoE-based framework to develop the digital twin model in Eq. (1).

In long-term operation, the underlying data distribution $\mathbb{P}_t$ may evolve due to changes in operating conditions, system dynamics, or environmental factors. Consequently, the predictive operator is not assumed to be static, but may evolve over time in response to distributional changes. This evolution is abstractly expressed as

$$\mathcal{T}_{\mathbb{P}_{t+1}} = \mathcal{U}(\mathcal{T}_{\mathbb{P}_t}, \mathbb{P}_{t+1}), \quad (2)$$

where $\mathcal{U}(\cdot)$ denotes a generic update mechanism driven by newly observed data. As the system operates and a distribution shift arises, the performance of a fixed data-driven digital twin can deteriorate. To address this issue, Sect. 3.3 introduces a transfer-learning-based updating approach.

3.2. Data-driven digital twin modeling through rolling-horizon prediction

This section presents the developed data-driven framework for digital twin modeling. Figure 1 provides an overview of the proposed framework. The framework consists of three successive phases. Phase I establishes a principled experimental design and data acquisition process to collect representative nominal operational data (Section 3.2.1). Phase II constructs a rolling-horizon data-driven digital twin by organizing multivariate measurements into finite observation-history vectors, forming temporally structured input–output samples, and training a predictive model for short-horizon forecasting (Sections 3.2.2 and 3.2.3). This predictive model serves as the core representation of the digital twin for online prediction and residual computation. In Phase III, online monitoring data are used to update the developed digital twin through transfer learning, as discussed in Sect. 3.3.

3.2.1. Data acquisition via design of experiments

The purpose of this phase is to design experiments and collect representative training data so that the training data fill as much as possible the task space. By doing so, we can ensure that the training data cover most of the tasks where test data are expected to be encountered in the real world. To achieve this goal, we adopt a Design of Experiments (DoE) strategy based on space-filling designs. Let $\mathcal{S} \subset \mathbb{R}^d$ denote the d -dimensional bounded operational space of

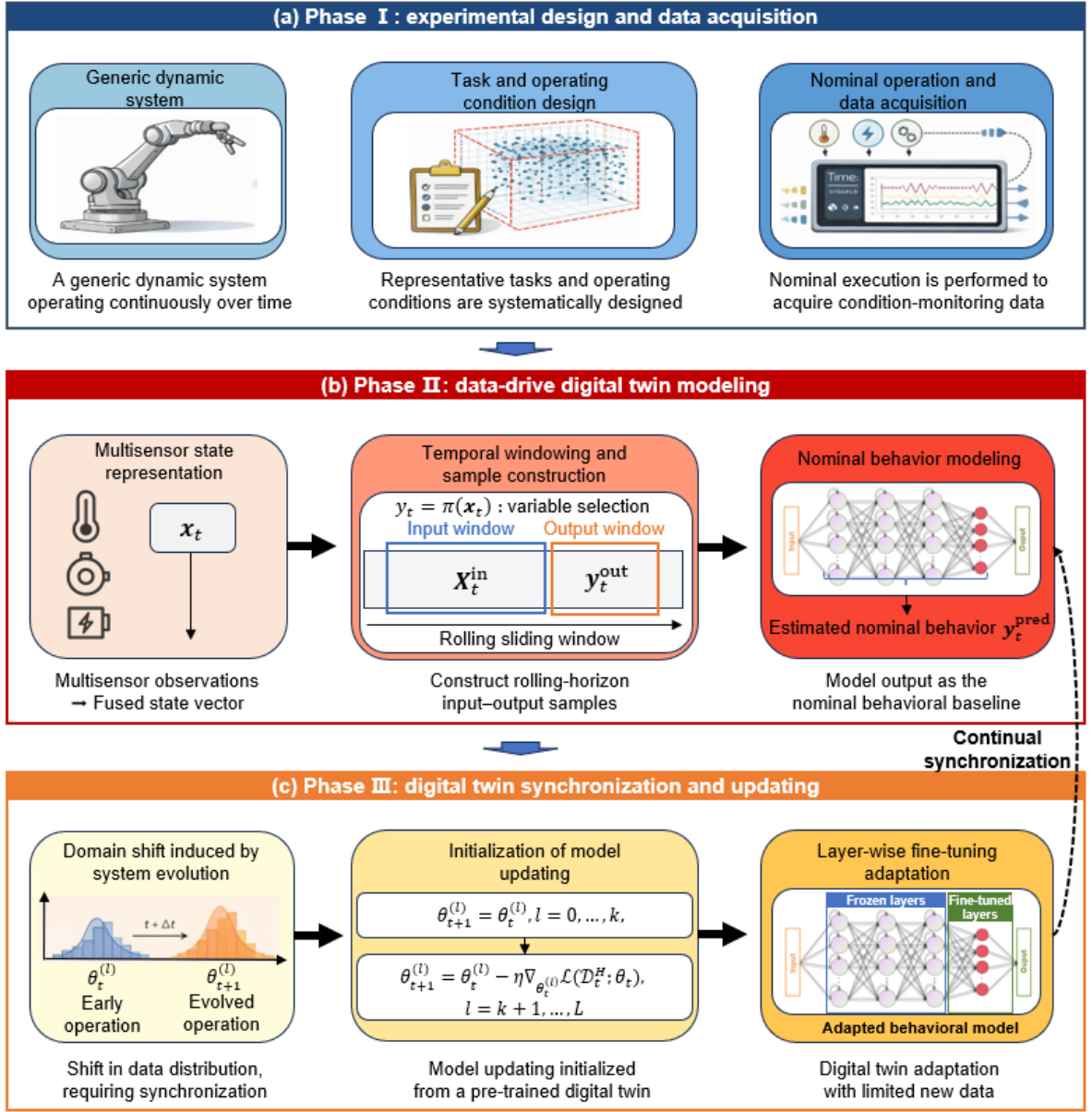


Figure 1: Overview of the proposed data-driven digital twin framework. Phase I establishes a principled experimental design and data acquisition layer, in which representative operating scenarios are systematically designed and executed to collect nominal condition-monitoring data from a generic dynamic system. Phase II constructs a data-driven digital twin by transforming multisensor observations into rolling-horizon input–output samples and learning a predictive model that serves as a nominal behavioral baseline. Phase III addresses long-term system evolution by continuously synchronizing and updating the digital twin in response to distributional shifts, enabling adaptive prediction under changing operating conditions.

the system, defined by physical constraints (e.g., joint limits, workspace boundaries). A task can, then, be represented as a time-dependent trajectory of points from the operational space $\mathcal{S}$. In this paper, we consider only a simple task of moving a system from a starting point $\mathbf{p}_0 \in \mathcal{S}$ to a goal point $\mathbf{p}_1 \in \mathcal{S}$ within a given duration t_{lk} . Further, we consider that all the tasks share the same duration t_{lk} and omit it for simplicity as it does not affect the task description. Let $\mathcal{S}_T = \mathcal{S} \times \mathcal{S}$ denote the task space of all the possible tasks. In practice, the start and goal configurations are sampled separately using two d -dimensional Halton sequences and are then paired to construct the task set. The objective of

the DoE is to generate an efficient set of N task configurations $\mathcal{P}_N = \{(\mathbf{p}_{0,i}, \mathbf{p}_{1,i})\}_{i=1}^N$ that maximizes uniformity and minimizes information redundancy, where $\mathbf{p}_{0,i} \in \mathcal{S}$ and $\mathbf{p}_{1,i} \in \mathcal{S}$ denote the separately sampled start and goal configurations, respectively.

Standard pseudo-random sampling often results in clustering and gaps due to probabilistic variance. Therefore, we employ a Low-Discrepancy Sequence (LDS), specifically the Halton sequence [44], which deterministically generates points that effectively fill the hypercube $[0, 1]^d$. The quality of the distribution is measured by the star discrepancy $D_N^*(\mathcal{P}_N)$, which quantifies the deviation of the empirical distribution from the ideal uniform distribution. Lower discrepancy implies better space-filling properties.

The Halton sequence is constructed using the radical inverse function. For a given prime base b , any non-negative integer i can be expanded as [44]:

$$i = \sum_{k=0}^M a_k(i)b^k, \quad (3)$$

where $a_k(i) \in \{0, 1, \dots, b-1\}$ are the digits of i in base b . The radical inverse function $\phi_b(i)$ mirrors these digits across the decimal point to the interval $[0, 1)$:

$$\phi_b(i) = \sum_{k=0}^M a_k(i)b^{-(k+1)}. \quad (4)$$

For both the start and goal configuration spaces, a d -dimensional Halton sequence is generated using the first d prime numbers, denoted as $b_1, b_2, \dots, b_d$. [44]. The i -th sample point $\mathbf{h}_i$ in the unit hypercube is defined as:

$$\mathbf{h}_i = (\phi_{b_1}(i), \phi_{b_2}(i), \dots, \phi_{b_d}(i)), \quad i = 1, \dots, N. \quad (5)$$

Since the bases are coprime, the sequence cycles at different rates for each dimension, ensuring that the points cover the d -dimensional space uniformly.

Finally, to obtain physical operating parameters, the points $\mathbf{h}_i$ are mapped from the unit hypercube to the physical constraints of the actual system. Let $\mathbf{l} = [l_1, \dots, l_d]^\top$ and $\mathbf{u} = [u_1, \dots, u_d]^\top$ be the lower and upper bounds of the operational parameters, respectively. The physical configuration point $\mathbf{x}_i \in \mathcal{S}$ is obtained via element-wise scaling:

$$\mathbf{x}_{i,j} = l_j + (\mathbf{h}_{i,j}) \cdot (u_j - l_j), \quad j = 1, \dots, d. \quad (6)$$

It should be noted that the Halton sequence is applied separately to the start and goal configuration spaces, and the sampled points are subsequently paired to form task configurations. The data-driven digital twin, however, predicts the dynamic behavior of the system response and therefore operates on a behavior space of the system's behavior, *e.g.*, positions temperatures and voltages of robot joints. Covering the task space does not necessarily mean good coverage of the entire behavior space and we do not claim this in the paper. However, there is usually a (near)-deterministic mapping between the task space and behavior space. Take the pick-up and place task for robots as an example. Each task can be represented by a starting position and an end position in the operation space. Once assigned, a given task configuration is converted into joint-space motion through inverse kinematics and trajectory planning algorithms, and the executed motion then generates the observed behaviors as a multivariate time-series. Because the mapping is fixed across all tasks, identical task configurations induce the same nominal behavior sequence. Therefore, by separately filling the start and goal configuration spaces with Halton samples and pairing them into tasks, we can obtain a representative sample of a region of the behavior space limited by commonly encountered tasks. Although we do not cover the entire behavior space, it is enough in this paper, as we only concern the performance of the digital twin on the commonly-encountered tasks.

3.2.2. Data preprocessing and sample construction

Following the problem formulation in Section 3.1, the multivariate time-series data collected under healthy operating conditions are transformed into supervised learning samples for rolling-horizon digital twin modeling. This step instantiates the abstract historical observation space into concrete input-output pairs suitable for model learning, while explicitly enforcing a temporal separation between input information and the reconstructed nominal trajectory.

Let $\{\mathbf{x}_t\}_{t=1}^T, \mathbf{x}_t \in \mathbb{R}^d$, denote the preprocessed multivariate observation sequence, where t indexes discrete time and $\mathbf{x}_t$ represents the system observation available at time t . For a decision at time t , a context window of length W_{in} ends W_{pred} steps before t , and the predictor directly reconstructs the intervening W_{pred} -step nominal output trajectory. When

a new observation becomes available, the window moves forward by one step and the same mapping is evaluated again. This construction creates an explicit temporal separation between the observations used as input and the endpoint at which the fault decision is made. Conceptually, the input features of the model can be defined by:

$$\mathbf{X}_t^{\text{in}} = [\mathbf{x}_{t-W_{\text{in}}-W_{\text{pred}}+1}, \dots, \mathbf{x}_{t-W_{\text{pred}}}]^{\top}, \quad (7)$$

The data used in the context window represents a finite observation history strictly preceding the target reconstruction window and therefore no information from the interval to be reconstructed is leaked.

The model prediction is

$$\mathbf{y}_t^{\text{out}} = [\hat{y}_{t-W_{\text{pred}}+1}, \dots, \hat{y}_t]^{\top}, \quad (8)$$

where $\hat{y}_{t'}$ denotes the digital twin's estimated target variable of interest at time t' . The prediction window therefore reconstructs the nominal behavior of the system over the most recent W_{pred} time steps, terminating at the current time t .

It should be noted that the model is used to generate a reference value for the subsequent fault detection task. Therefore, having a good prediction performance is not enough. We should look for a model that could reliably and robustly generate reference values that support accurate fault detection. This will affect the choice of W_{pred} . In a typical single-step forecasting scheme, we set $W_{\text{pred}} = 1$. Under the indexing convention adopted in Eqs. 7–8, the context window ends at $t - 1$ and the model predicts the response at time t . Since adjacent observations are often strongly correlated, a simple persistence-like prediction $\hat{y}_t \approx y_{t-1}$ may already achieve high prediction accuracy without necessarily capturing sufficiently informative temporal dynamics. However, when the trained model is used to generate reference values for the fault detection model, such a reference value will lead to biased fault detection results. Using a multi-step prediction scheme with $W_{\text{pred}} > 1$ can alleviate this problem by increasing the temporal separation between the latest input observation and the current decision point.

Another advantage of the multi-step framework is that, compared with single-step, multi-step prediction separates the context window from the current decision point. With $W_{\text{pred}} = 10$, the estimate $\hat{y}_t$ is produced without directly using measurements from the ten samples immediately preceding t . A single-step predictor would place the input window adjacent to the decision point; early fault-affected observations could therefore enter the context before the current residual is evaluated and cause the predictor to partially follow the abnormal trend. The separated multi-step window provides a less contaminated nominal reference, at the cost of a longer prediction horizon. Section 6.1 evaluates this trade-off over multiple history and prediction lengths.

Each supervised learning sample thus consists of an input–output pair $(\mathbf{X}_t^{\text{in}}, \mathbf{y}_t^{\text{out}})$ that is temporally aligned with the same current time index t . This construction enables rolling-horizon prediction, in which the digital twin repeatedly reconstructs the nominal trajectory associated with each successive system state as new observations become available. Prior to sample construction, standard preprocessing steps including data cleaning and normalization are applied to ensure temporal consistency and numerical stability. The input-window length W_{in} and reconstruction-window length W_{pred} are selected on the validation set. The sensitivity analysis in Section 6.1 shows that increasing the history length does not yield a monotonic benefit and supports the retained setting $W_{\text{in}} = 30$, $W_{\text{pred}} = 10$.

3.2.3. Predictive model learning

This section describes the modeling of a data-driven digital twin as a concrete realization of the predictive operator defined in Section 3.1. The digital twin provides a continuous and synchronized virtual representation of the physical system under healthy operating conditions. Rather than acting as a standalone predictive model, it is formulated as a virtual entity that evolves in parallel with the physical system by continuously assimilating observations and generating predictions of expected nominal behavior.

Based on the input–output samples constructed in Section 3.2.2, the abstract predictive operator associated with the nominal data distribution at time t , denoted by $\mathcal{T}_{\mathbb{P}_t}$, is instantiated through a parameterized nonlinear mapping

$$\mathcal{T}_{\mathbb{P}_t} \equiv f_{\text{DT}}(\cdot; \theta_t), \quad (9)$$

where θ_t denotes the parameter set of the digital twin at the current operating time t . Accordingly, for each current time index t , the nominal behavior reconstruction aligned with the rolling-horizon window is given by

$$\mathbf{y}_t^{\text{pred}} = f_{\text{DT}}(\mathbf{X}_t^{\text{in}}; \theta_t), \quad (10)$$

where $\mathbf{X}_t^{\text{in}}$ denotes the finite observation history representing the most recent system behavior and $\mathbf{y}_t^{\text{pred}}$ corresponds to the reconstructed nominal behavior over a finite horizon. Under the rolling-horizon scheme, the same parameterized mapping is repeatedly evaluated as the context window moves forward, yielding a time-indexed sequence of nominal reconstructions aligned with system evolution. All data used for model training are collected under healthy operating conditions, ensuring that the learned digital twin represents nominal system behavior.

From a modeling perspective, the digital twin maintains a persistent virtual–physical correspondence by recursively including new observations in the context window. This continuous information updating allows the virtual entity to remain synchronized with the physical system timeline and to generate nominal reconstructions that are temporally aligned with real system operation. Such synchronized nominal predictions provide a consistent virtual reference against which deviations of the physical system can be used to design fault detection methods in Section 4.

It should be noted that we use a deep learning model for $f_{DT}(\cdot)$. The θ_t here thus represents the internal state parameters (weights) of the deep learning model. The input to the $f_{DT}(\cdot)$ is the observed covariates in a past window. It is important to distinguish the internal parameters of the deep network and its inputs from the state variables in the state space-based or behavior modeling framework. The hidden variables in the deep learning models are internal learned features rather than the physical state of the robot. However, consistent with the neural state-space modeling paradigm, they form a learned latent representation induced from data that serves as a task-specific proxy of the system state for nominal prediction [45]. The same principle that a deep network can discover a low-dimensional latent representation in which the dominant dynamics become tractable is also reported in the deep Koopman embeddings of nonlinear systems [46]. Similarly, we do not claim that the inputs need necessarily be the state variables. As long as trained properly within a deep learning framework, the inputs of historical observations can be used to learn a low-dimensional representation of the system dynamics [45, 46].

In this work, $f_{DT}(\cdot; \theta_t)$ is implemented by a stacked long short-term memory (LSTM) network followed by a fully connected output layer. The network receives a 30×18 observation history and directly outputs the ten-step motor-temperature trajectory. LSTM gating equations are standard and are therefore omitted; the architecture and training settings required for reproduction are reported in Table 5. The proposed acquisition, adaptation, residual, and PROF procedures do not require an LSTM specifically. Section 6.1 screens LSTM, Transformer, and XGBoost predictors under the same data split and input–output definition. LSTM is retained because it provides competitive nominal accuracy, the lowest measured inference latency, the smallest model footprint, and direct compatibility with the layer-wise parameter-transfer experiment. Thus, the paper does not claim a new LSTM architecture; the methodological contribution is the integrated digital-twin-driven detection framework.

3.3. Data-driven digital twin updating through transfer learning

During long-term operation, the physical system undergoes gradual changes because of aging, parameter drift, and changing operating conditions. Although such evolution may not immediately impair system functionality, it progressively alters nominal dynamics and creates a distribution shift relative to the training data. A fixed digital twin can therefore develop a growing virtual–physical mismatch that appears as systematic residual drift. Transfer learning is used to update the nominal model and restore consistency with the current verified healthy operating domain.

Let θ_0 denote the parameter set of the initial data-driven digital twin model. As the system evolves, a limited amount of newly observed operational data $\mathcal{D}_t^H$ becomes available at the current operating time t , reflecting the stage-dependent nominal behavior of the physical system. The objective of digital twin updating is to obtain an updated parameter set θ_t such that the predictive operator induced by the digital twin remains consistent with the current physical system.

For most data-driven digital twins based on deep neural networks, the parameters at time t can be organized based on layers:

$$\theta_t = \left\{ \theta_t^{(0)}, \theta_t^{(1)}, \dots, \theta_t^{(L)} \right\}, \quad (11)$$

where lower-indexed layers typically encode system-invariant temporal dynamics, while higher layers capture variations in nominal behavior induced by long-term operation. For an LSTM-based digital twin, this layered organization naturally corresponds to stacked recurrent layers followed by output layers. In this paper, digital twin updating uses the standard transfer-learning operations of parameter freezing and fine-tuning. Depending on target-data availability, lower recurrent layers can be frozen while higher layers and the output mapping are adapted, or all parameters can be fine-tuned. The freezing operation itself is not claimed as a new continual- or incremental-learning algorithm; it is the implementation used to maintain the nominal reference within the proposed detection framework.

As a representative example, when the first k layers are frozen during updating, the parameter evolution from time t to $t + 1$ is expressed as

$$\theta_{t+1}^{(l)} = \theta_t^{(l)}, \quad l = 0, \dots, k, \quad (12)$$

$$\theta_{t+1}^{(l)} = \theta_t^{(l)} - \eta \nabla_{\theta_t^{(l)}} \mathcal{L}(\mathcal{D}_t^H; \theta_t), \quad l = k + 1, \dots, L, \quad (13)$$

where η denotes the learning rate and $\mathcal{L}(\cdot)$ is a prediction loss evaluated using healthy operational data from the current stage.

It should be noted that in this study, “source” and “target” denote two confirmed healthy operating domains with the same prediction task and sensing definition but different nominal distributions; they do not denote different robot control tasks. Transfer learning is performed when a new device or a verified healthy operating stage defines such a target domain, rather than continuously at every sampling instant. More specifically, the updating follows a *verify-then-update* policy. We distinguish among different cases:

- Initial adaptation to a new device: a new device may differ from the source device in its initial condition, sensor bias, and thermal or dynamic characteristics. Therefore, a small amount of target-domain data is first collected under controlled and confirmed healthy conditions and used to adapt the source digital twin.
- Normal continuous operation: after the initial adaptation, the model parameters remain fixed during online monitoring. Slow and continuous changes that can still be represented by the recent observation history may be naturally tracked by the rolling prediction process and therefore do not require immediate parameter updating. This is controlled by checking the residuals from the prediction model and skipping the update if the residuals are within a safe range.
- Persistent model–system mismatch: if system evolution exceeds the representative capability of the current model, the residual may increase persistently or exceed a threshold for model updating. After verifying the cause of the deviation,
 - if the deviation is caused by a fault or incipient abnormality, the corresponding data are excluded from model updating and maintenance or fault-handling actions are required;
 - if the deviation is confirmed to result from healthy long-term evolution, such as normal aging, recalibration, component replacement, or maintenance-induced changes, these data are used for model updating through transfer learning.

To avoid the transfer learning being activated by model uncertainty or incipient failure, we first use the developed fault detection model to screen the incoming data for potential failures. Only when the incoming data is confirmed by the fault detection model to be a normal sample, will they be used for transfer learning. In this way, we can safely avoid the contamination from faulty data in transfer learning. Incipient failure or early-phase degradation, however, are difficult to exclude as the faulty signal can be too weak to trigger a failure alarm. Even in this case, updating the model with these weak degradation or incipient failure signals would not impact too much the model performance as the faulty signals are weak and will not change dramatically the behavior of the model. Therefore, the *verify-then-update* policy allows updating system nominal behavior while avoiding impacts from the failure signals.

The “verify-then-update” policy for transfer learning, however, has a potential risk: some normal drifts might be erroneously excluded by the policy as their patterns confuse the fault detector since both of them exhibit a deviation from the reference values. In this paper, we make an assumption that the drift from normal operation should be gradual and in the early phase, it would not trigger an alarm from the fault detector. We also acknowledge that this is an assumption that is difficult to verify in practice. In the future, we could consider developing a refined fault detector so that it can differentiate the drift from normal operation from the early failure signals.

4. Fault detection through prediction-based recursive observation feedback from digital twins

This section develops a fault detection framework grounded in the predictive digital twin model introduced in Section 3. Section 4.1 establishes the residual-based decision mechanism. Section 4.2 introduces the fault-detection algorithm that avoids amplifying errors during rolling-horizon operation. Section 4.3 extends the framework to cross-domain scenarios via transfer-learning-based generalization.

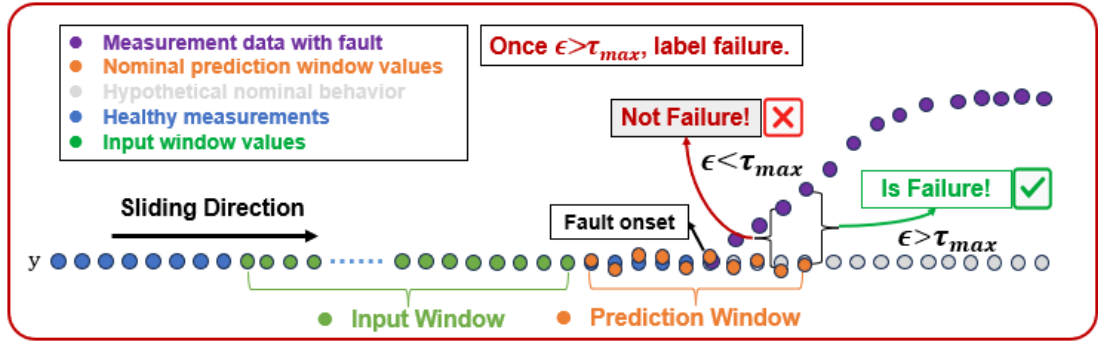


Figure 2: Principles of residual-based fault detection

4.1. Residual-based fault detection

Rather than relying on explicit fault models or labeled fault samples, the proposed fault detection approach is based on the residual between real system observations and the nominal behavior reconstructed by the digital twin. Let y_t denote the measured value of the target variable at the current time index t , and let $\hat{y}_t$ denote the corresponding nominal value produced by the digital twin at time t . As defined in Section 3.2, at each current time index t the digital twin reconstructs a nominal trajectory aligned with the reconstruction window $[t - W_{\text{pred}} + 1, t]$. In the proposed fault detection setting, however, the binary decision is made *only* at the window end, i.e., based on the residual at the current point t .

Let y_t^{raw} denote the immutable measurement stored by the monitoring system. The point-wise residual used for every fault decision is

$$\epsilon(t) = |\hat{y}_t - y_t^{\text{raw}}|, \quad (14)$$

where $|\cdot|$ denotes the absolute value. The resulting residual can then be used as an input feature for fault detection. The rationale is that under normal operating conditions, $\hat{y}_t$ remains consistent with y_t^{raw} and $\epsilon(t)$ stays small. When abnormal behavior or faults occur, the discrepancy between nominal reconstruction and measurement increases, leading to larger $\epsilon(t)$. In the literature, various approaches have been used for fault detection based on the reconstructed residual error, including threshold-based methods where the reconstruction errors are directly compared with a threshold [47], statistical methods, where the reconstructed errors are used to design a statistical hypothesis testing for fault detection [48] and machine learning/deep learning methods, where the reconstructed errors are used as feature to train a machine learning/deep learning model for fault detection [49].

In this paper, we adopt a threshold-based approach for fault detection. As illustrated in Fig. 2, a threshold-based binary decision rule is adopted to map the point-wise residual into a fault indicator at time t :

$$C(\epsilon(t)) = \begin{cases} 1, & \epsilon(t) > \tau_{\text{res}}, \\ 0, & \epsilon(t) \leq \tau_{\text{res}}, \end{cases} \quad (15)$$

where τ_{res} is a residual threshold, and $C(\cdot)$ denotes a residual-based fault detector.

In the implemented study, the threshold τ_{res} is treated as a hyper-parameter through tuning in a validation set from the source domain. To support the hyper-parameter tuning, we need a few labeled data that are injected through simulation in the source domain. The hyper-parameter tuning is done through optimizing the F1-score:

$$\tau_{\text{res}}^* = \arg \max_{\tau_{\text{res}} \in \mathcal{T}} \text{F1-score}(\tau_{\text{res}}), \quad (16)$$

where $\mathcal{T}$ denotes a candidate threshold set. Once determined, τ_{res}^* is fixed and reused in the target domain without target-domain fault labels or re-optimization. Consequently, model training and domain adaptation through transfer learning require healthy data only. We only need a few injected faults in the source domain to support the threshold

tuning. Although it is not completely label-free, it can greatly reduce the required labeled data. Especially, we do not require labeled data from the target domain.

In theory, a normal-data-only alternative can also be tested: one could set the threshold as an upper quantile or a statistical confidence bound of the healthy residuals. However, if we do not use labeled fault signals at all, we can only ensure the precision of the detector, but cannot control the recall as we have no information at all on the failed samples. How to achieve balanced detection performance without labeled data remains a challenging issue that deserves future research.

It should be noted that the developed fault detection framework relies on the digital twins to create input features. However, a good prediction model does not necessarily guarantee reliable fault detection. In particular, a reliable and robust fault detection model faces the following challenges:

1. the prediction of the digital twins should be robust against the contamination in the historical observations brought by failure signals and create robust reconstructed residuals. We address this issue through the developed PROF algorithm in Sect. 4.2;
2. the reconstructed residual errors should remain robust against potential shifts from varying operation conditions. In the developed framework, we address this challenge through transfer learning, which is presented in detail in Sect. 4.3.

Overall, the developed fault detection model combines the nominal predictor from the data-driven digital twin, a validation-calibrated residual rule, transfer-based nominal adaptation, and PROF. Their individual contributions are evaluated through ablation studies in Section 6.2.

4.2. The PROF algorithm

Under rolling-horizon operation, the digital twin repeatedly forms new input sliding windows as time advances. As shown in Fig. 3 (a), if a fault-affected measurement is reused as an input, the nominal reconstruction can progressively follow the abnormal trajectory, reducing subsequent residuals and increasing missed detections. To avoid this contamination effect, we develop a new algorithm called fault detection with prediction-based recursive observation feedback (the PROF). The key idea of the PROF algorithm is explained in Fig. 3 (b). **PROF addresses the contamination problem while preserving data authenticity by maintaining two separate sequences: the immutable raw monitoring record $\{y_t^{\text{raw}}\}$ used for residual calculation and fault decisions, and an internal buffer recording the data used in the sliding windows for performance prediction $\{\tilde{y}_t\}$.** After the raw observation has been evaluated, the internal buffer is updated as

$$\tilde{y}_t = \begin{cases} \hat{y}_t, & \epsilon(t) > \tau_{\text{res}}, \\ y_t^{\text{raw}}, & \epsilon(t) \leq \tau_{\text{res}}. \end{cases} \quad (17)$$

Due to the sliding-window construction introduced in Section 3.2, the corrected internal value affects only future windows whose historical segments contain time t . For a future decision index $t' > t$, the target-variable entries of the effective input window are formed from the feedback buffer:

$$\mathbf{X}_{t'}^{\text{in}} = [\dots, \tilde{y}_{t'-W_{\text{pred}}}, \dots], \quad (18)$$

while the decision at t' remains based on $y_{t'}^{\text{raw}}$. PROF therefore never overwrites the archived signal and cannot remove a persistent fault from the decision stream: each new raw sample is evaluated before any feedback-buffer update.

Fig. 3 illustrates the motivation and effect of the proposed recursive feedback strategy under rolling-horizon operation. After fault onset, fault-affected measurements may not immediately trigger a failure decision if the resulting point-wise residual remains below the detection threshold. Without recursive feedback, as shown in Fig. 3(a), once a fault is detected at time t , the corresponding faulty measurement is directly rolled into subsequent input windows as the prediction horizon advances. This causes the digital twin to gradually follow the faulty trajectory, leading to error propagation and a reduction in point-wise residuals at later decision instants, which may compromise detection reliability. With PROF, as shown in Fig. 3 (b), once a fault is detected, the faulty measurement at time t is replaced by its nominal reconstruction before being incorporated into future input windows. This replacement prevents the contamination of historical inputs, stabilizes the rolling-horizon reconstruction of nominal behavior, and preserves the sensitivity of residual-based fault detection over time. For clarity of illustration, Fig. 3 shows the rolling-horizon

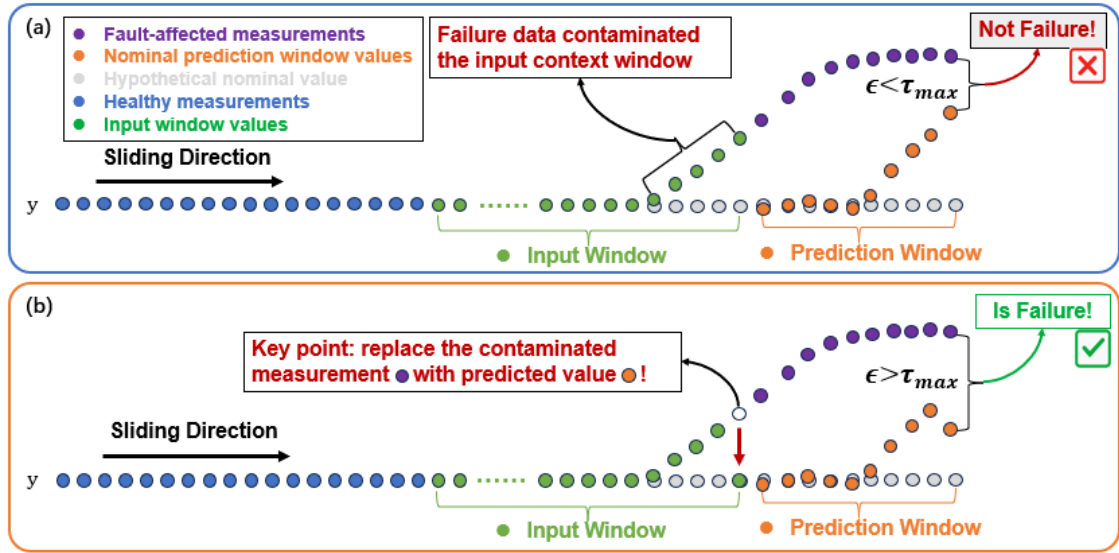


Figure 3: Illustration of the PROF mechanism under rolling-horizon operation. (a) Without PROF, fault-affected measurements are rolled into subsequent input windows, causing error propagation and suppression of point-wise residuals in later decision steps. (b) With PROF, once a fault is detected at time t , the fault-affected measurement is replaced by its nominal prediction, preventing error propagation and preserving the sensitivity of residual-based fault detection.

operation along the target variable y only, while the actual digital twin operates on multivariate input sequences $\mathbf{x}_t$. The gray points indicate hypothetical nominal values that would be observed in the absence of faults and are shown for conceptual reference only; they are not observable in practice and are not used by the proposed method.

The activation boundary of PROF can be stated formally. For a fault beginning at t_f , define the first detection time

$$t_d = \min\{t \geq t_f : \epsilon(t) > \tau_{res}\}. \quad (19)$$

Before t_d , abnormal observations are indistinguishable from nominal prediction uncertainty under the current hard decision rule since their signals are too weak and may temporarily enter the feedback buffer. PROF does not retrospectively remove them. From t_d onward, newly detected abnormal values are replaced before the next rolling input is formed, suppressing further recursive contamination. If a weak fault remains below threshold for its entire duration, PROF will not be activated and the fault may be missed. Avoiding this limitation would require a probabilistic or soft-decision extension, with an associated false-alarm trade-off. However, since the fault signals are still weak (otherwise they will be corrected by the PROF), their impacts on the reconstructed reference values will not be very significant. We therefore choose to accept these potential small contamination and design PROF to only correct the more severe faults.

Overall, PROF integrates (i) point-wise residual-based fault decision at the window end and (ii) recursive observation feedback at the same decision index. This yields a lightweight and robust fault detection mechanism compatible with rolling-horizon digital twin operation, without requiring explicit fault models or labeled fault samples during deployment.

4.3. Generalization through transfer learning

In this paper, we calibrate the decision threshold using source domain validation data, and directly apply it on the target domain. It should be mentioned that this direct reuse can be done only when the monitored quantity, sensing resolution, sampling rate, preprocessing and normalization, residual definition, prediction horizon, and fault decision semantics remain unchanged. In addition, the target nominal model must be adapted with verified healthy data so that its fault-free residual scale is comparable with the source residual scale. In particular, we should be able to demonstrate that after the transfer learning on the predictive digital twin, the distributions of the residual errors between the reconstructed reference values and the actual measurements are close enough so that a threshold calibrated based on the residual distribution on the source domain can be safely applied on the target domain. These conditions hold for the two ArmPi

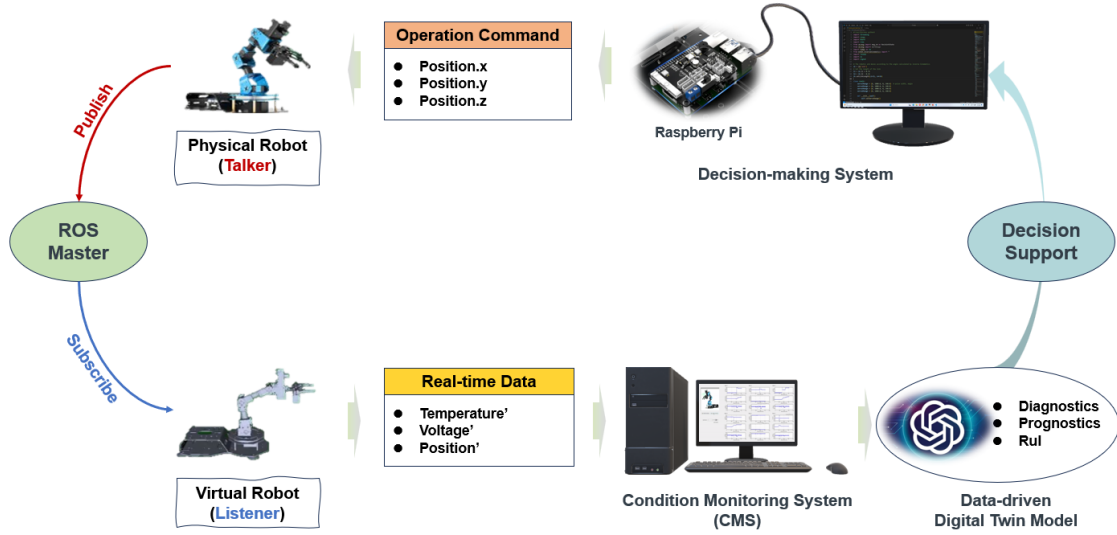


Figure 4: Overview of the robotic digital twin platform and its components.

robots used as case study in this paper. Section 6.1 verifies the domain shift and shows that transfer learning concentrates target residuals around zero; Section 6.2.1 reports the threshold-sensitivity curve. Threshold reuse is not asserted for sensors, sampling protocols, fault modes, or operating envelopes that violate these conditions.

An above-threshold residual establishes a model–system inconsistency, but does not by itself identify its physical cause. The proposed operational logic therefore separates detection from diagnosis. A transient or persistent exceedance first triggers an abnormal-state flag and quarantines the associated observations from any model update. Maintenance evidence, additional sensors, or controlled inspection is then required to distinguish (i) a fault or incipient abnormality, for which the model remains fixed and fault-handling action is taken, from (ii) verified healthy evolution or model inadequacy, for which a new healthy batch may be collected and the digital twin updated at low frequency. This conservative logic prevents suspected-fault data from being learned as nominal behavior, while acknowledging that fully automatic fault-versus-drift diagnosis is beyond the present residual detector.

If the aforementioned prerequisite does not hold, especially, when the residual distributions differ significantly between the source and target domains, transfer learning can be applied again to update the optimal values of the detection threshold.

5. Experiment design

The proposed digital twin modeling and fault detection framework is validated on a real robotic platform. Section 5.1 introduces the experimental setup and system configuration. Section 5.2 describes the digital twin modeling implementation. Section 5.3 presents a study demonstrating the strength of the DoE over traditional random sampling. Section 5.4 details the fault detection experiments based on the PROF mechanism.

5.1. Experimental robotic digital twin platform

To support the development and experimental validation of the proposed data-driven digital twin framework, an open-source robotic demonstration platform was used. The platform integrates a ROS-based physical robotic system with a Simulink-based virtual robotic model, enabling real-time condition monitoring, data-driven digital twin modeling, and decision support. This platform provides the experimental foundation for the modeling, updating, and evaluation methods introduced in the subsequent sections. An overview of the platform architecture is shown in Fig. 4, and its main components are described below.

A **six-degree-of-freedom ArmPi robotic arm** serves as the physical experimental platform in this study, as illustrated in Fig. 5. The robot is compatible with ROS, with each joint driven by a smart serial bus servo motor. These actuators provide real-time feedback on joint position, temperature, and voltage, resulting in a total of 18 sensor

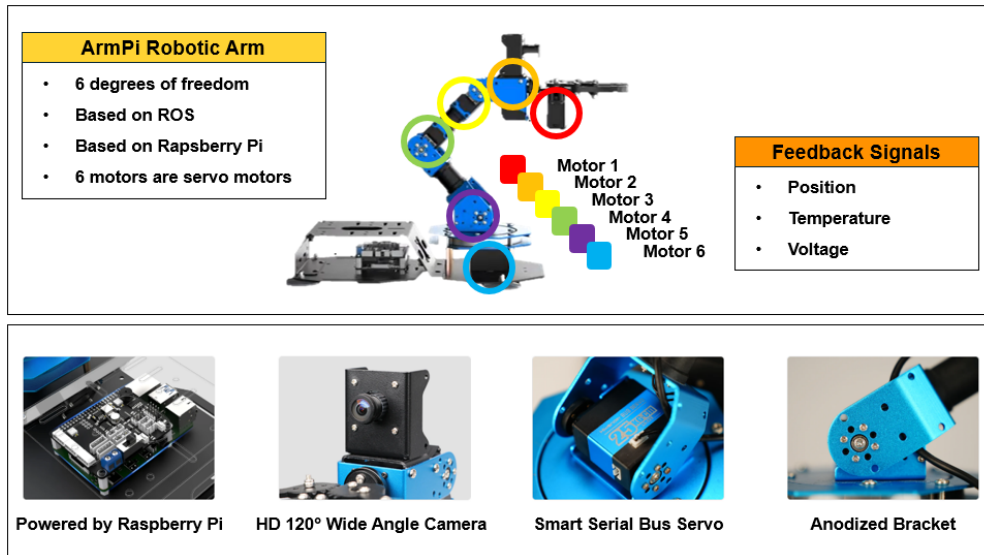


Figure 5: Structure and components of the ArmPi robotic arm.

Table 1

Condition monitoring parameters.

Parameter Name	Symbol	Unit	Description
Temperature	T	$^{\circ}\text{C}$	Motor temperature during operation.
Voltage	V	V	Motor working voltage.
Position	P	/	Joint position of the motor.

signals. The robotic arm executes pick-and-place operations and trajectory tracking tasks based on inverse kinematics control.

A **condition monitoring system (CMS)**, implemented to acquire and process multisensor data from the physical robot via ROS. The CMS subscribes to joint position, motor temperature, and motor voltage signals, assigns synchronized timestamps, and supports subsequent digital twin modeling. The monitored parameters are summarized in Table 1. All source code and datasets generated using this experimental platform are publicly available at: https://github.com/sonic160/digital_twin_robot.

5.2. Data-driven digital twin modeling

In this study, to validate the data-driven digital twin modeling and updating framework introduced in the previous sections, two robots in Fig. 5 with different levels of motor degradation, were configured as experimental platforms. We train a data-driven digital twin on one robot (referred to as source robot), and test the model updating through transfer learning on the other robot (referred to as target robot). This configuration serves as a controlled experimental setting for validating the proposed digital twin construction, rolling horizon prediction, and transfer learning based updating strategy. The corresponding task structure, sampling strategy, and data collection process that implement the experiment design discussed earlier are illustrated in Fig. 6.

To enable the collection of diverse and representative data for model development, a set of pick and place tasks was systematically designed to generate representative operational data. As illustrated in Fig. 6, each task consists of five sequential movements: returning to the initial position, moving to the pick up location, grasping the object, transporting it to the placement location, and releasing the object. For each task, the pick up and placement positions are predefined, and the corresponding joint trajectories are computed through inverse kinematics to ensure physical feasibility. During execution, sensor data including motor position, voltage, and temperature are collected at a frequency of 10 Hz, providing time series measurements of system behavior under various task conditions.

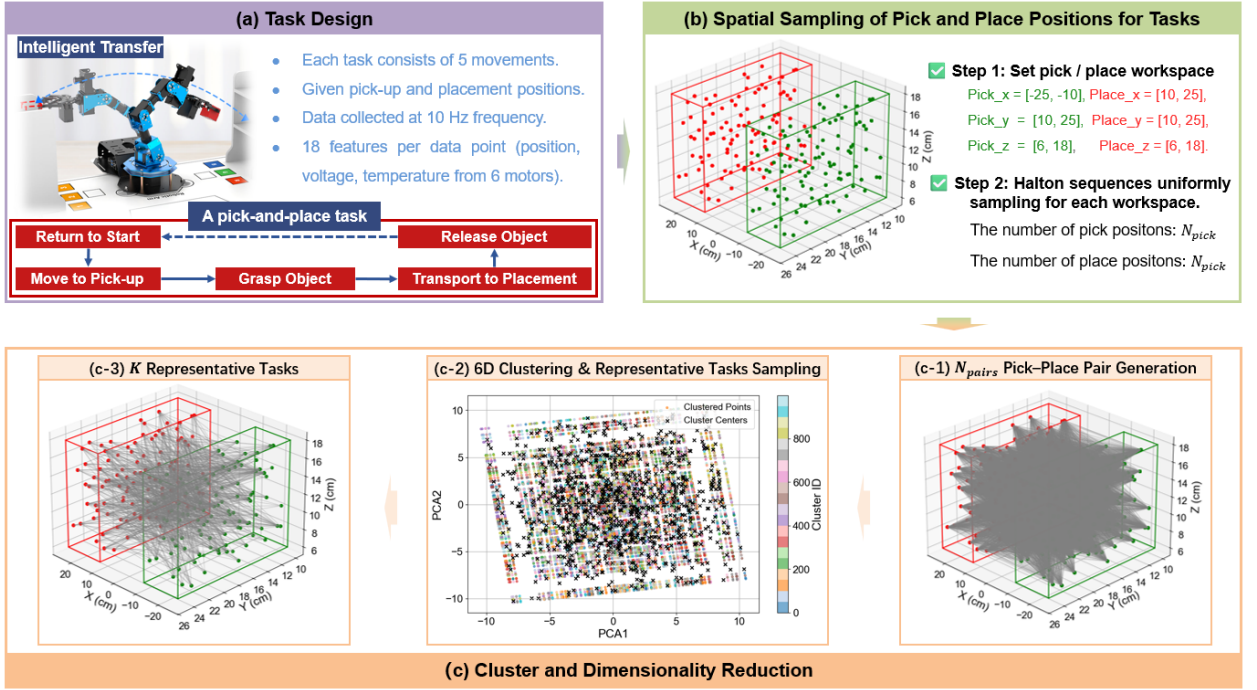


Figure 6: Task Design and Sampling Strategy for Robotic Pick and Place Experiments

The sampled space is the predefined feasible Cartesian pick-and-place envelope, rather than the complete dynamic state space of the robot. As shown in Fig. 6(b), its spatial boundaries were defined from workspace and safety constraints. A Halton low-discrepancy design generated $N_{\text{pick}} = 100$ pick points and $N_{\text{place}} = 100$ place points. Executing the resulting feasible trajectories then produced diverse joint-level temporal responses. The design is intended to improve representative task-space coverage under a fixed sampling budget; it does not guarantee exhaustive coverage of every operating condition.

By connecting all possible combinations of the 100 pick positions and 100 place positions, a total of $100 \times 100 = 10,000$ candidate pick place task pairs were generated. Each pair was validated through inverse kinematics to ensure feasibility, resulting in $N_{\text{pairs}} = 10,000$ valid task trajectories. As illustrated in Fig. 6(c), a clustering based sampling strategy was then applied to reduce redundancy and enhance diversity. Each task pair was encoded as a six-dimensional vector based on the spatial coordinates of the pick and place positions. MiniBatch K Means clustering was performed to partition the 10,000 task samples into $K = 1,270$ clusters. To facilitate visualization and diversity assessment, the high dimensional vectors were projected into a 2D space using principal component analysis (PCA). From each cluster, a representative task was selected by identifying the sample closest to the cluster center, resulting in a final set of 1,270 representative pick place trajectories that uniformly span the task space and are used for subsequent data collection and model training.

The source and target robots executed the selected representative pick and place tasks, generating the corresponding source domain dataset D_{src} and target domain dataset D_{tgt} , respectively. A total of $K_{\text{src}} = 1000$ tasks and $K_{\text{tgt}} = 270$ tasks were performed by the source and target robots, ensuring diversity while maintaining task consistency across platforms. During task execution, the condition monitoring system (CMS) collected 18 dimensional multivariate time series data including position, temperature, and voltage signals from six motors at a sampling rate of 10 Hz. This resulted in a total of 10,400 seconds (i.e., 104,000 data points) of data for the source robot and 1,404 seconds (i.e., 14,040 data points) for the target robot, yielding two platform specific datasets: $D_{\text{src}} \in \mathbb{R}^{104000 \times 18}$ and $D_{\text{tgt}} \in \mathbb{R}^{14040 \times 18}$. To support model development and evaluation, the datasets were partitioned into training, validation, and testing subsets without shuffling to preserve temporal dependencies. Specifically, D_{src} was split into $D_{\text{src}}^{\text{train}}$, $D_{\text{src}}^{\text{val}}$, and $D_{\text{src}}^{\text{test}}$.

and $D_{\text{src}}^{\text{test}}$ using a ratio of 7:1.5:1.5. Similarly, D_{tgt} was divided into $D_{\text{tgt}}^{\text{train}}$ and $D_{\text{tgt}}^{\text{test}}$ with a ratio of 8:1, as summarized in Table 2.

Table 2

Summary of dataset partitions and fault injection statistics

Dataset Name	Domain	Type	Total Points	Total Samples	Description
$D_{\text{src}}^{\text{train}}$	Source	Normal	72,800	72,761	Source domain training set (normal data)
$D_{\text{src}}^{\text{val}}$	Source	Normal	15,600	15,561	Source domain validation set (normal data)
$D_{\text{src}}^{\text{test}}$	Source	Normal	15,600	15,561	Source domain test set (normal data)
$D_{\text{tgt}}^{\text{train}}$	Target	Normal	12,480	12,441	Target domain training set (normal data)
$D_{\text{tgt}}^{\text{test}}$	Target	Normal	1,560	1,521	Target domain test set (normal data)
$D_{\text{src}}^{\text{val-fault}}$	Source	Fault-injected	15,600	15,561	Source domain validation set with fault-injected data
$D_{\text{src}}^{\text{test-fault}}$	Source	Fault-injected	15,600	15,561	Source domain test set with fault-injected data
$D_{\text{tgt}}^{\text{test-fault}}$	Target	Fault-injected	1,560	1,521	Target domain test set with fault-injected data

5.3. Task-space coverage and domain-shift characterization

The performance of the DoE through Halton sequences is evaluated at the stage where it acts directly: we evaluate the coverage of normalized Cartesian task coordinates under a given sample budget. We use centered discrepancy D_c (smaller is better) to measure multidimensional deviation from a uniform coverage and average two-dimensional projected grid coverage C_{2D} (larger is better) to measure how broadly pairwise projections occupy the feasible region. We choose random sampling as a baseline as it is widely-used to generate training data in the current data-driven digital twin studies. Random sampling is repeated with three independent seeds; the deterministic Halton design is reported once. Table 3 shows substantially lower discrepancy and higher projected coverage for both pick and place regions. This supports the proposed methods as a space-filling task design.

Table 3

Task-space coverage under an identical sampling budget. Random results are mean $\pm$ SD over three seeds.

Method	Region	Centered discrepancy $\downarrow$	Avg. 2D coverage $\uparrow$
Random	Pick	0.005294 ± 0.003099	0.6322 ± 0.0259
Random	Place	0.007692 ± 0.003922	0.6256 ± 0.0195
Halton	Pick	0.000271	0.7567
Halton	Place	0.000271	0.7567

To quantify the source-to-target shift, Table 4 compares motor 6 measurements. Temperature and voltage exhibit large standardized mean differences and Kolmogorov–Smirnov (KS) statistics, whereas motor position remains closely matched. The two robots therefore execute comparable motion configurations but differ markedly in electrical and thermal responses, which motivates nominal-model adaptation rather than attributing the target degradation solely to a different task distribution.

The LSTM defined in Section 3.2.3 learns a data-driven digital twin from the source-domain training data $D_{\text{src}}^{\text{train}}$ and is subsequently evaluated and adapted in the target domain. Consistent with the sample construction above, the predictor estimates motor 6 temperature from the 18 measured covariates. The developed methods and baselines are compared using R^2 , MAPE, RMSE, and MSE in Section 6.1.

Table 4

Statistical comparison of motor 6 variables between source and target domains.

Indicator	Temperature (°C)	Voltage	Position
Source mean $\pm$ SD	40.13 $\pm$ 1.48	7246.56 $\pm$ 47.90	644.54 $\pm$ 172.33
Target mean $\pm$ SD	43.25 $\pm$ 1.84	7404.93 $\pm$ 38.67	642.21 $\pm$ 167.52
Mean difference	+3.12	+158.38	-2.33
KS statistic	0.671	0.937	0.029
Standardized difference	1.87	3.64	-0.014

Figure 8 illustrates the joint distribution of motor 6 position, voltage, and temperature in the source and target domains. Compared to the source domain, the target domain exhibits a higher concentration of temperature values in the upper range and a distinct shift in voltage position correlation. Such discrepancies may contribute to the degradation of model generalization when transferred across domains, and they provide an empirical basis for evaluating the digital twin updating strategy in the subsequent analysis.

5.4. Fault injection and detection through the PROF algorithm

In this work, we inject faults related to abnormal temperature rise through simulation to test the developed fault detection methods. Simulation is chosen instead of physical fault injection mainly because injecting real faults risks damaging the robots and raises safety and budget concerns. To model the temperature response of an electric motor under an abnormal temperature rise such as that caused by rotor stall, we adopt a first-order lumped-parameter thermal network (LPTN) [50, 51, 52, 53]. Based on this model, transient thermal faults were synthetically injected into the temperature signals of motor 6 using a semi-empirical formulation to simulate realistic degradation scenarios (see Eq. (23)). The thermal dynamics are governed by:

$$C_{th} \cdot \frac{dT(t)}{dt} + \frac{T(t) - T_{env}}{R_{th}} = P(t) \quad (20)$$

where $T(t)$ is the system temperature, T_{env} is the ambient temperature, C_{th} is the thermal capacitance, R_{th} is the thermal resistance, and $P(t)$ is the thermal power input. The thermal time constant is defined as:

$$\tau = R_{th} \cdot C_{th}$$

Assuming a constant power input $P(t) = P_{fault}$ during the fault period, Equation (20) has the analytical solution:

$$T(t) = T_{\infty} - (T_{\infty} - T_0) e^{-t/\tau}, \quad \text{where } T_{\infty} = T_{env} + R_{th} \cdot P_{fault} \quad (21)$$

Here, T_0 is the initial temperature, and T_{∞} is the theoretical steady state temperature.

After fault removal at time $t = t_{cool}$, the power input drops to zero, i.e., $P(t) = 0$. The system then enters a natural cooling process governed by:

$$T(t) = T_{env} + (T_{peak} - T_{env}) e^{-(t-t_{cool})/\tau} \quad (22)$$

where $T_{peak} = T(t_{cool})$ is the temperature at the moment the fault is repaired. The full temperature profile $T(t)$, including both heating and cooling stages, can be expressed as a piecewise function:

$$T(t) = \begin{cases} T_{\infty} - (T_{\infty} - T_0) e^{-t/\tau}, & t < t_{cool} \\ T_{env} + (T_{peak} - T_{env}) e^{-(t-t_{cool})/\tau}, & t \geq t_{cool} \end{cases} \quad (23)$$

The injected failure signals represent a physically-motivated progressive thermal anomaly rather than a complete electromechanical simulation of a specific failure. An additional heat input produces first-order heating and its removal produces exponential cooling, consistent with the dominant temperature consequence of temporary overload, increased mechanical friction, restricted cooling, or short-duration jamming. The construction is preferred to an arbitrary step or random perturbation because onset, amplitude, duration, and recovery remain interpretable and repeatable.

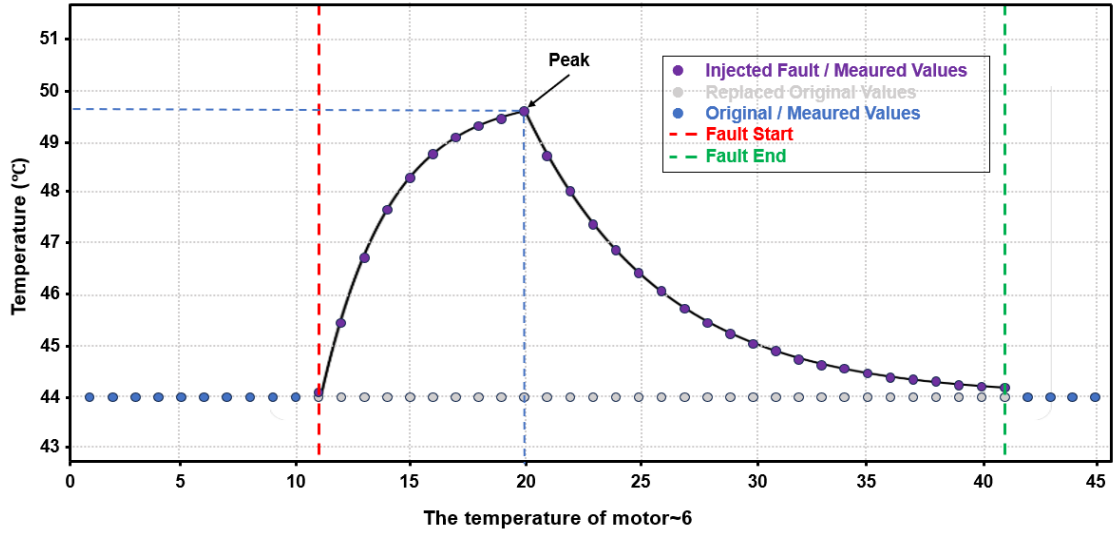


Figure 7: Synthetic motor temperature trajectory based on LPTN

Faults spanning 30–50 samples and reaching 4–8 °C above the nominal trajectory were inserted into the source validation and test sets and the target test set. With the 1 °C sensor resolution, a point was labeled faulty only when the injected trajectory exceeded its baseline by more than 1 °C. The saved evaluation files contain 676 labeled decision points in each source fault subset and 164 in $D_{\text{tgt}}^{\text{test-fault}}$ after rolling-window alignment. The training of the data-driven digital twins and its adaptation to the target domain used only fault-free data. The threshold was calibrated only on $D_{\text{src}}^{\text{val-fault}}$ and then fixed.

It should be noted that real motor failures can additionally involve time-varying load, speed, ambient conditions, nonlinear heat transfer, controller intervention, and electrical, mechanical, or vibration signatures that are not represented by the first-order LPTN. The present experiment therefore validates detection of transient or gradually developing overheating anomalies, not every motor failure mode or root-cause diagnosis. Non-thermal, highly intermittent, or temperature-unobservable faults lie outside the current validation boundary; naturally occurring or safely reproduced physical faults remain future work. However, the developed framework remains applicable to real motor failures, as long as we can collect training data from real experiments. In this case, we just need to replace the injected failure with the actual failure data.

6. Results and discussions

This section presents a comprehensive evaluation of the proposed framework. Section 6.1 first analyzes the performance of data-driven digital twin modeling in both source and target domains. Section 6.2 then investigates residual-based fault detection performance, including threshold calibration, transfer learning effects, and the factorial ablation of PROF and transfer learning. **Section 6.3 reports inference latency and target-domain fine-tuning time.**

6.1. Data-driven digital twin modeling performance

The objective of this section is to assess how accurately the data-driven digital twin can model the nominal thermal behavior of motor 6, and if the developed transfer learning strategy can update the digital twin model to adapt to changes.

Following the data-driven digital twin modeling procedure described in Section 3, the multivariate observation of the system at discrete time t is denoted by the vector $\mathbf{x}_t$. The target variable y_t is extracted from the multivariate observation via a projection operator $\pi(\cdot)$, i.e., $y_t = \pi(\mathbf{x}_t)$. In the experimental setting considered in this work, the observation vector $\mathbf{x}_t$ is constructed by combining temperature, joint position, and working voltage measurements collected from six motors, resulting in an 18-dimensional multivariate representation. In this work, the target variable y_t corresponds to the temperature of motor 6.

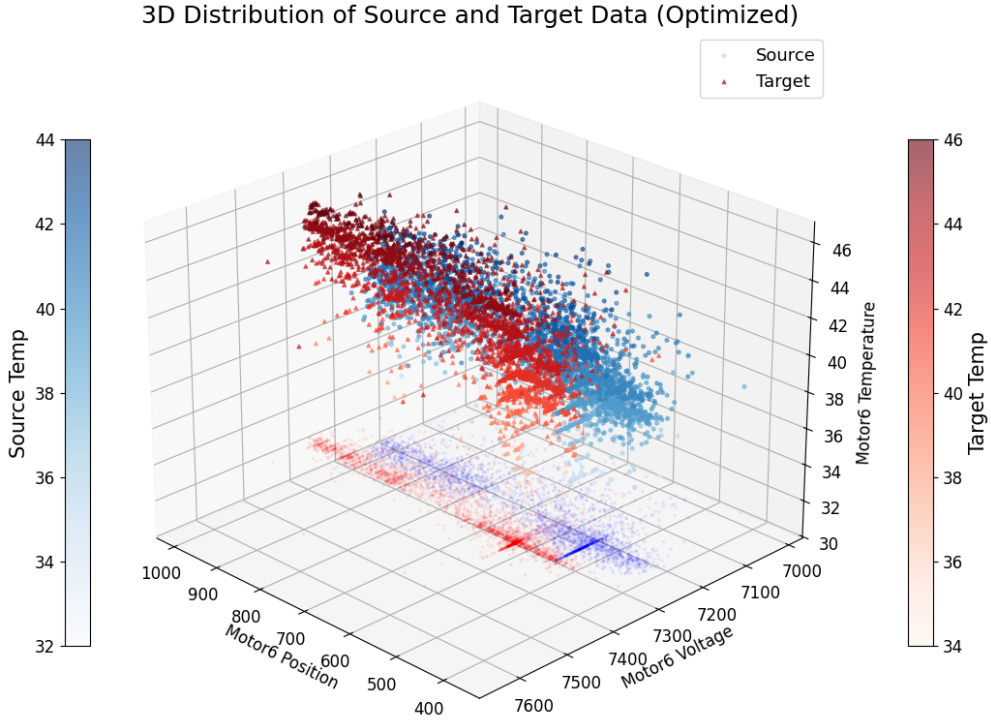


Figure 8: Cross Domain Distribution of Motor 6 Signals

Supervised learning samples are constructed using a sliding-window strategy. For each sample aligned with the current time index t , the input context window $\mathbf{X}_t^{\text{in}}$ consists of an ordered sequence of historical observations preceding the prediction interval. Under the experimental parameter setting adopted in this study, the input window contains 30 consecutive time steps and the prediction lead time is 10. Therefore, we have $\mathbf{X}_t^{\text{in}} = [\mathbf{x}_{t-39}, \dots, \mathbf{x}_{t-10}]^T$.

The corresponding output window $\mathbf{y}_t^{\text{pred}}$ represents the nominal trajectory of the target variable aligned with time t , corresponding to the digital twin's nominal estimation over the most recent 10 time steps: $\mathbf{y}_t^{\text{pred}} = [\hat{y}_{t-9}, \dots, \hat{y}_t]^T$. Since the target variable y_t is instantiated as the temperature of motor 6, the above definition can be equivalently written as the 10-step nominal temperature reconstruction $\mathbf{y}_t^{\text{pred}} = [\hat{T}_6(t-9), \dots, \hat{T}_6(t)]^T$. As each sample spans 40 consecutive time steps, the total number of available samples is reduced by 39 relative to the number of raw time points in the dataset. The resulting sample counts for all datasets are summarized in Table 2.

A three-layer Long Short-Term Memory (LSTM) network is used to model the nonlinear temporal relationships in the multivariate system dynamics. The network architecture and training hyperparameters are summarized in Table 5. In all experiments, early stopping based on validation loss is employed to prevent overfitting.

6.1.1. Source-domain nominal modeling

We first evaluate the digital twin modeling performance in the source domain. The LSTM-based digital twin is trained using the source-domain training dataset $\mathcal{D}_{\text{src}}^{\text{train}}$, validated on $\mathcal{D}_{\text{src}}^{\text{val}}$, and finally evaluated on the held-out test dataset $\mathcal{D}_{\text{src}}^{\text{test}}$.

Before subsequent transfer-learning and fault-detection experiments, the selection of the forecasting backbone was examined under a controlled protocol. The LSTM is compared with two other commonly-used machine learning regression models, *i.e.*, Transformer and XGBoost. All the three models were trained using all 104,000 source-domain observations, the same chronological 70%/15%/15% split, the same 18 variables, a 30-step history, a direct 10-step output, and the same held-out test set. Neural models used batch size 64, at most 100 epochs, patience 10, dropout 0.2,

Table 5
Model hyperparameters and descriptions

Parameter Name	Value	Description
Input_dim / d	(30, 18)	The number of input features for each time step in LSTM.
Hidden_dim / h	256	The number of hidden units in the LSTM layer.
Output_dim	(10, 1)	The number of future time steps to predict.
Num_layers	3	The number of stacked LSTM layers in the model.
Dropout	0.2	Dropout rate between LSTM layers to prevent overfitting.
Epochs	100	The maximum number of training epochs.
Learning_rate	0.0001	Learning rate for Adam optimizer.
Loss_function	MSELoss	Loss between predictions and ground truth.
Optimizer	Adam	Optimizer used to update model parameters.

Adam, and MSE loss. XGBoost received the identical 30×18 history flattened to 540 features and predicted the same ten outputs; only algorithm-specific optimization settings differed.

Table 6
Controlled source-domain nominal-model comparison on the held-out test set (seed 42).

Model	RMSE	R^2	MAPE (%)	Train (s)	Size (MB)
LSTM	0.4411	0.8591	0.7550	147.5	0.36
XGBoost	0.4239	0.8699	0.7235	36.7	24.38
Transformer	0.4430	0.8579	0.7651	598.8	0.43

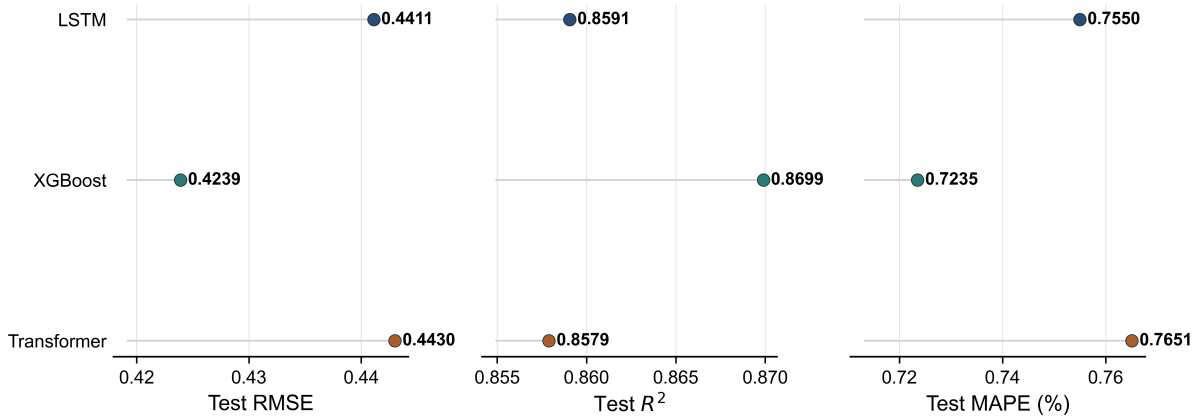


Figure 9: Source-domain nominal-model accuracy comparisons.

Table 6 and Fig. 9 show that XGBoost gives the best point estimate, with an R^2 advantage of 0.0108 over the archived LSTM checkpoint, but its model is substantially larger. LSTM requires 147.5 s to train, longer than XGBoost (36.7 s) but substantially shorter than Transformer (598.8 s). Transformer has a similar point estimate to LSTM. LSTM is retained because it combines competitive accuracy with the smallest footprint, the lowest batch-1 inference latency reported in Section 6.3, a moderate offline training cost, and direct compatibility with the layer-wise parameter-transfer procedure. The archived LSTM checkpoint is used consistently throughout the source prediction, direct-generalization, and transfer-learning analyses; XGBoost and Transformer are not carried into downstream adaptation. The framework itself is independent of the forecasting architecture and can use another temporal predictor, provided that it supports rolling prediction and generates the nominal reference required for residual calculation and PROF.

We also tested history lengths $W_{in} \in \{20, 30, 40, 50, 60\}$ with prediction horizons $W_{pred} \in \{10, 15, 20, 25, 30\}$. Figure 10 shows that a longer history does not monotonically improve either R^2 or MSE. At the retained horizon $W_{pred} = 10$, prediction performance is stable across the tested history lengths and the 30-step setting is near-optimal. We therefore use the fixed setting $W_{in} = 30$ and $W_{pred} = 10$ for all models, preserving a consistent temporal structure without introducing the additional complexity of adaptive window selection.

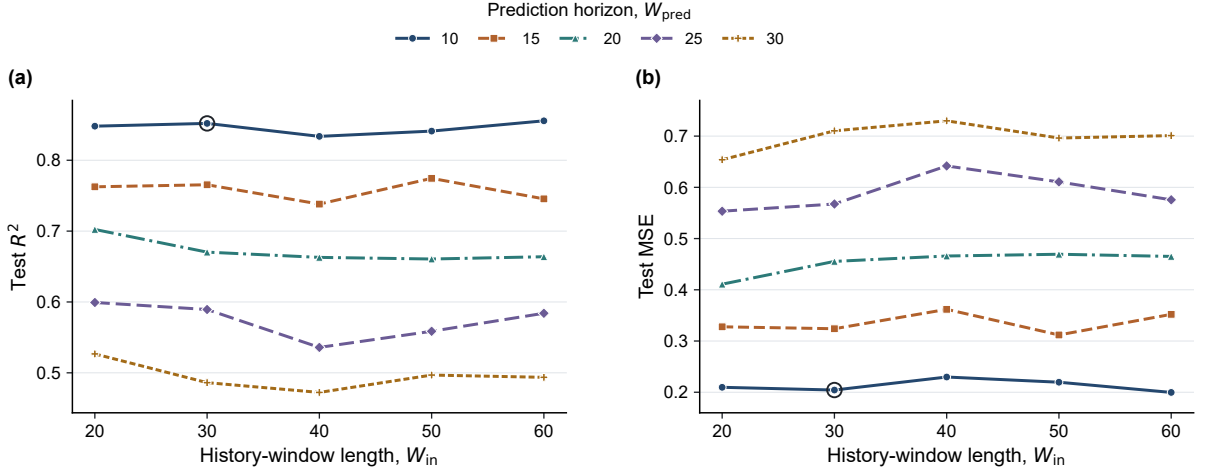


Figure 10: Sensitivity of source-domain test performance to history-window length under different prediction horizons: (a) R^2 and (b) MSE. The outlined point denotes the retained $W_{in} = 30, W_{pred} = 10$ setting.

The prediction results in the source domain are shown in Fig. 11. Using the archived LSTM checkpoint subsequently employed for direct source-to-target deployment and transfer learning, the digital twin achieves $R^2 = 0.8824$, MSE = 0.1826, RMSE = 0.4273, and MAPE = 0.724% on the validation set. On the held-out test set, the corresponding values are $R^2 = 0.8591$, MSE = 0.1946, RMSE = 0.4411, and MAPE = 0.755%. Thus, Fig. 11, Table 6, and the source-reference curve in Fig. 13 now report the same archived LSTM checkpoint.

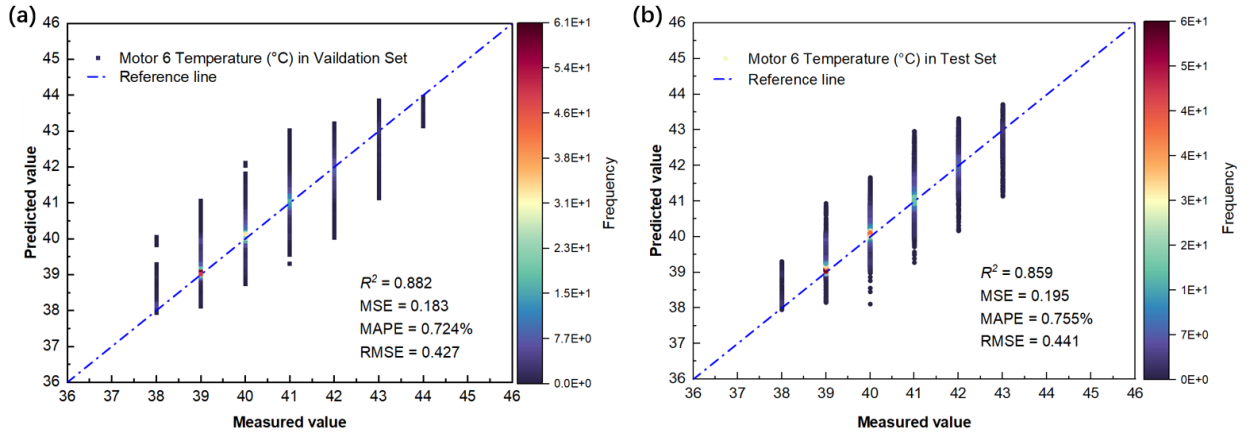


Figure 11: Prediction performance of the LSTM-based digital twin in the source domain. (a) Validation results. (b) Test results.

6.1.2. Updating the digital twin through transfer learning

On the target robot operating under degraded conditions, a series of training scenarios were constructed to systematically evaluate the adaptability of the source-domain digital twin and the effectiveness of different updating

strategies, as summarized in Table 7. In all experiments, the target-domain test set was kept fixed with 1,521 samples, while the number of target-domain training samples used for model updating was gradually increased from 12.5% (1,521 samples) to 100% (12,441 samples). This experimental setting ensures a fair comparison of updating performance under different target-domain data availability, while eliminating the influence of variations in the test set.

Table 7

Training configurations in the target domain (after sample construction). Test sets are fixed.

Scenario	Usable Training Samples	Test Samples
1	1,521 (12.5%)	1,521
2	3,081 (25%)	
3	4,641 (37.5%)	
4	6,201 (50%)	
5	7,761 (62.5%)	
6	9,321 (75%)	
7	10,881 (87.5%)	
8	12,441 (100%)	

Based on this setup, a transfer-learning-based updating strategy is further evaluated, in which the target digital twin is initialized using the parameters learned from the source system and subsequently fine-tuned using healthy target-domain data. Following the digital twin updating mechanism introduced in Section 3.3, this transfer process is implemented via layer-wise fine-tuning of a three-layer LSTM predictor. The recurrent layers are indexed as layers 0, 1, and 2 from bottom to top. Different updating strategies are defined by selectively freezing lower layers while fine-tuning higher layers and the output fully connected (FC) layer, as follows:

- Freeze LSTM layer 0; fine-tune layers 1–2 and the output FC layer;
- Freeze LSTM layers 0–1; fine-tune only layer 2 and the output FC layer;
- Freeze all LSTM layers (layers 0–2); fine-tune only the output FC layer;
- No layers frozen; fine-tune all LSTM layers (layers 0–2) and the output FC layer.

For experimental comparison, the source-trained digital twin is first directly deployed to the target domain as a *direct generalization* baseline. In addition, a *target-only learning* configuration is considered, where the model is trained from scratch using only target-domain data. Together with the layer-wise transfer updating strategies, these configurations enable a systematic comparison of digital twin modeling performance under degradation. The nominal temperature prediction results on the fault-free target-domain validation and test sets are shown in Fig. 12.

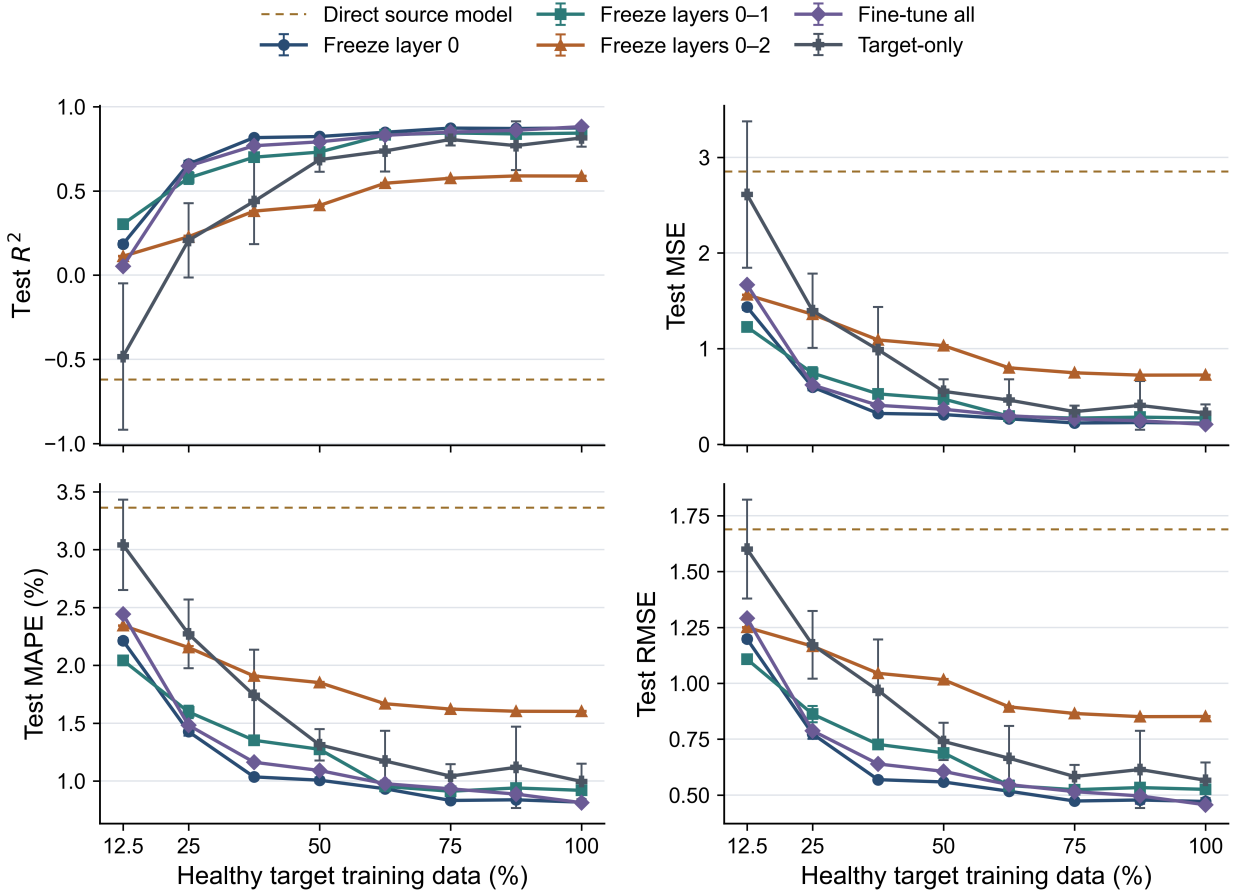


Figure 12: Nominal modeling performance on the target test set $\mathcal{D}_{\text{tgt}}^{\text{test}}$ under different layer-freezing configurations. Curves show the mean and error bars show one sample standard deviation over five independent seeds.

It can be observed that direct generalization performs poorly across all evaluation metrics, with R^2 values close to zero or even negative, and substantially higher MSE, RMSE, and MAPE compared to the updated models. This indicates that the system dynamics learned from the source robot are no longer sufficient to accurately characterize the thermal behavior of the degraded target robot without updating, experimentally confirming the degradation-induced cross-domain failure of a static digital twin. In contrast, target-only learning exhibits gradual performance improvement as more target-domain training samples become available, but remains consistently inferior to the transfer-based updating methods over the entire range of sample ratios, particularly under limited data availability, highlighting the limited data efficiency of learning from scratch.

As the proportion of target-domain training data increases, all transfer-based updating strategies exhibit a consistent performance improvement trend. Specifically, when the training sample ratio reaches approximately 25%–50%, the predictive performance is substantially recovered, with R^2 increasing rapidly to above 0.6, and RMSE and MAPE reduced by more than half compared to the direct generalization baseline. When the sample ratio further increases beyond 75%, the performance gains gradually saturate and additional improvements become marginal. These results demonstrate that even a limited amount of target-domain data is sufficient to effectively mitigate degradation-induced modeling errors through digital twin updating, highlighting the strong data efficiency of the proposed framework in the target domain.

A further comparison among different layer-freezing configurations reveals that the choice of frozen layers has a significant impact on updating performance. Among all strategies, freezing only the lowest layer (*Freeze layer 0*) achieves the best or near-optimal performance across almost all target-domain sample ratios. In the low-to-medium

sample regime (12.5%–50%), this configuration consistently yields higher R^2 values and lower MSE, RMSE, and MAPE than the other strategies, while maintaining stable performance as the sample size increases. In contrast, fully fine-tuning all network parameters tends to result in degraded or unstable performance when target-domain data are limited, and target-only learning fails to match the performance of transfer-based updating under the same data conditions. Based on these observations, *Freeze layer 0* is adopted as the default updating strategy for all subsequent target-domain digital twin updating experiments.

Here, the target-only baseline is trained from random initialization using only healthy target-domain data; it contains no source parameters and is therefore the “no transfer learning” condition in the component ablation. For the residual-alignment analysis in Fig. 13, the pre-transfer baseline is the same archived source LSTM reported in Fig. 11 and Table 6, deployed on the target test set without any target-domain parameter update. The post-transfer curve is generated from this checkpoint after freezing LSTM layer 0 and fine-tuning the remaining layers on 100% of the designated healthy target training split, with validation-based early stopping. The direct-generalization value in Figs. 12 and 13 is identical, and the two target curves in Fig. 13 use the same 1,521 test windows and differ only by transfer adaptation. Direct source-to-target deployment yields $\text{MSE} = 2.8525$ and $R^2 = -0.6200$, whereas transfer learning reduces the MSE to 0.2170 and raises R^2 to 0.8768. Moreover, the Wasserstein distance between the target and source residual distributions decreases from 1.588 to 0.157, and the Kolmogorov–Smirnov statistic decreases from 0.820 to 0.221. These results show that transfer learning substantially reduces the prediction error toward the source reference and therefore provide empirical support, rather than a universal guarantee, for reusing the source-calibrated residual threshold under the sensing, preprocessing, and operating conditions stated in Section 4.3. The target-only results remain in the ablation comparison because they answer a different question: whether transferred initialization outperforms learning solely from target-domain data.

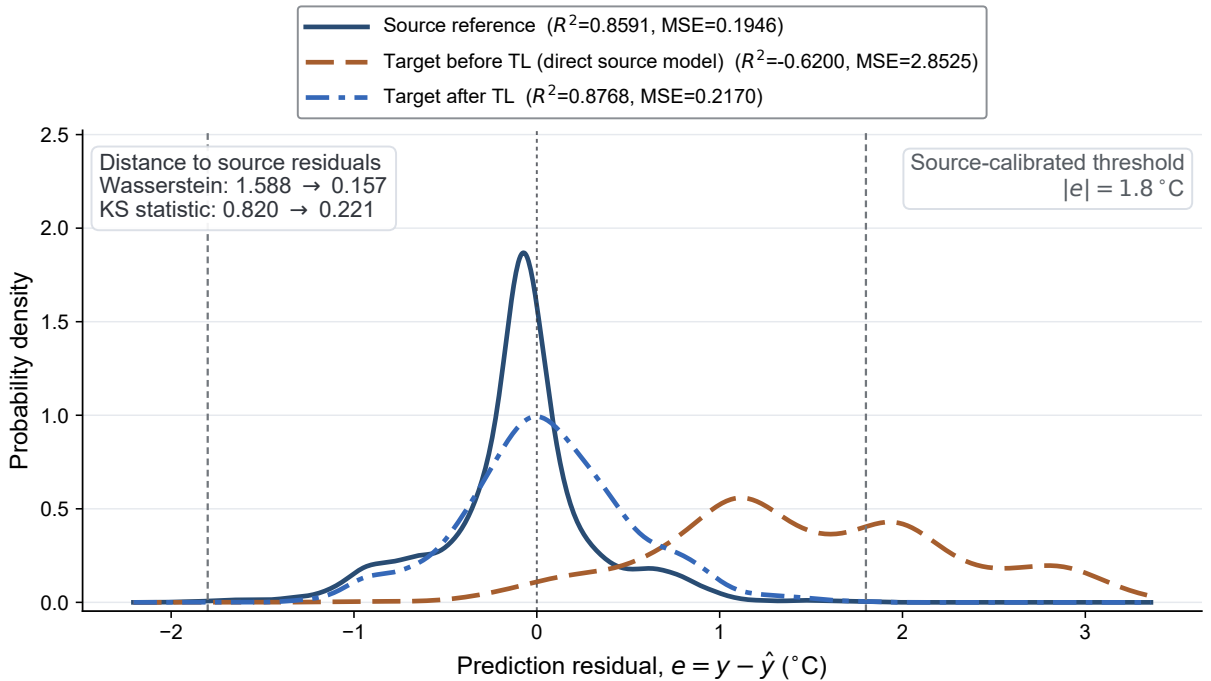


Figure 13: Controlled residual-density comparison for the original transfer experiment underlying Fig. 12. The solid blue curve shows source-domain residuals; the red dashed curve shows the baseline obtained by applying the source-trained LSTM directly to the target test set; and the blue dashed curve shows target-domain residuals after transfer learning. Both target curves use the identical 1,521 test windows. The vertical dashed lines indicate the source-calibrated decision boundary $|e| = 1.8^\circ\text{C}$.

6.2. Residual-based fault detection performance

6.2.1. Threshold calibration

This subsection addresses threshold calibration for the residual-based fault detection framework introduced in Section 4. Under rolling-horizon operation, the digital twin predicts motor temperature using healthy operational data, and fault detection is performed by comparing prediction residuals against a decision threshold. All experiments employ digital twins trained exclusively with healthy data as described in Section 3.3.

Motor 6 temperature is selected as the monitored variable. The point-wise residual $\epsilon(t)$ is computed according to Eq. (14). To prevent information leakage, the residual threshold τ_{res} is calibrated solely on the fault-injected source validation set $\mathcal{D}_{\text{src}}^{\text{val-fault}}$. All source fault subsets are transformed using the feature and target scalers saved with the trained LSTM checkpoint; the scalers are not refitted during threshold calibration or testing.

A grid search is conducted over $\tau_{\text{res}} \in [0, 6]$ with step size 0.1, and the optimal threshold is selected by maximizing the F1-score (Eq. (16)). Figure 14 presents the resulting F1-score curve. A clear unimodal trend is observed, with the maximum F1-score of 0.8949 achieved at $\tau_{\text{res}} = 1.8$ when PROF is enabled. Smaller thresholds increase false positives due to noise sensitivity, whereas larger thresholds suppress informative residual deviations and increase missed detections. Accordingly, $\tau_{\text{res}} = 1.8$ is fixed for all subsequent experiments on both source and target systems without re-calibration. This choice explicitly uses labeled injected faults in the source validation set. The deployment advantage is narrower: no target-domain fault labels or deliberate target-robot fault experiments are required. Cross-domain reuse is supported by the identical sensing and preprocessing pipeline and by the transfer-induced residual alignment in Fig. 13, but is not claimed outside those conditions. It should be noted that in this paper, we calibrate the threshold only on the source domain validation data and directly use it on the target domain. This direct application works as long as the distributions of the residual errors between model predictions and observations are sufficiently similar in the source and target domains. This condition may hold after transfer learning when the residual distributions of the two domains become sufficiently aligned.

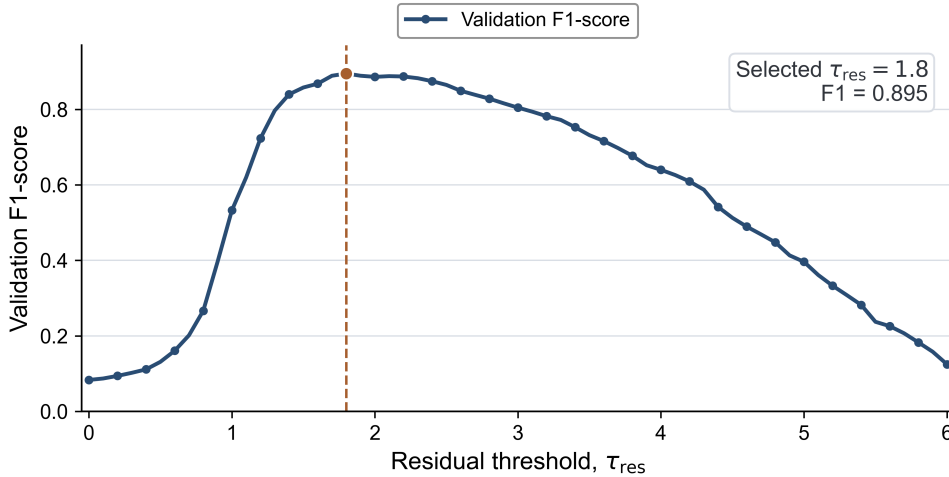


Figure 14: F1-score versus residual threshold on the fault-injected source validation set $\mathcal{D}_{\text{src}}^{\text{val-fault}}$; the selected operating point is explicitly identified.

6.2.2. Performance on the source domain

With the threshold fixed at $\tau_{\text{res}} = 1.8$, residual-based fault detection is evaluated on the source system. The digital twin trained on healthy source data generates rolling-horizon predictions, and a binary decision is made based on the absolute residual at the final prediction step. A sample is classified as faulty when the residual exceeds τ_{res} .

Table 8 summarizes the results on $\mathcal{D}_{\text{src}}^{\text{val-fault}}$ and $\mathcal{D}_{\text{src}}^{\text{test-fault}}$. On the validation set, the method achieves an accuracy of 0.9911 and an F1-score of 0.8949. On the unseen source-domain test set, it achieves an accuracy of 0.9904 and an F1-score of 0.8837. The small decrease on the test set indicates that the validation-selected threshold generalizes consistently to unseen source-domain trajectories.

Although accuracy remains high due to the dominance of normal samples, recall and F1-score provide a more informative assessment of fault sensitivity. Precision remains above 0.91 on both subsets, demonstrating effective control of false alarms while retaining more than 0.84 recall on the unseen test trajectories.

Table 8

Residual-based fault detection performance on the source domain

Dataset	Accuracy	Precision	Recall	F1-score
$\mathcal{D}_{src}^{val-fault}$	0.9911	0.9150	0.8757	0.8949
$\mathcal{D}_{src}^{test-fault}$	0.9904	0.9283	0.8432	0.8837

6.2.3. Impact of transfer learning on target-domain detection

The source-calibrated threshold $\tau_{res} = 1.8$ is directly applied to the target system without re-optimization. For each target training ratio in Table 7, two digital twins are constructed: (i) a transfer-based model initialized from source parameters and fine-tuned with healthy target data, and (ii) a target-only model trained from scratch. In both cases, fault detection follows the same residual construction and rolling-horizon scheme.

Figure 15 presents detection performance on the target fault test set $\mathcal{D}_{tgt}^{test-fault}$. At the smallest 12.5% target-training ratio, the two approaches have nearly identical mean F1-scores (0.4433 for transfer learning and 0.4414 for target-only learning), but the transferred model has substantially lower variability across seeds. From 25% onward, transfer learning provides substantially higher accuracy, precision, recall, and F1-score at all intermediate ratios. At the full-data endpoint, the transferred model retains higher accuracy, precision, and F1-score, whereas target-only learning has slightly higher recall. Overall, the results support faster and more stable adaptation once a minimal amount of verified healthy target data is available, without claiming uniform superiority in every individual metric.

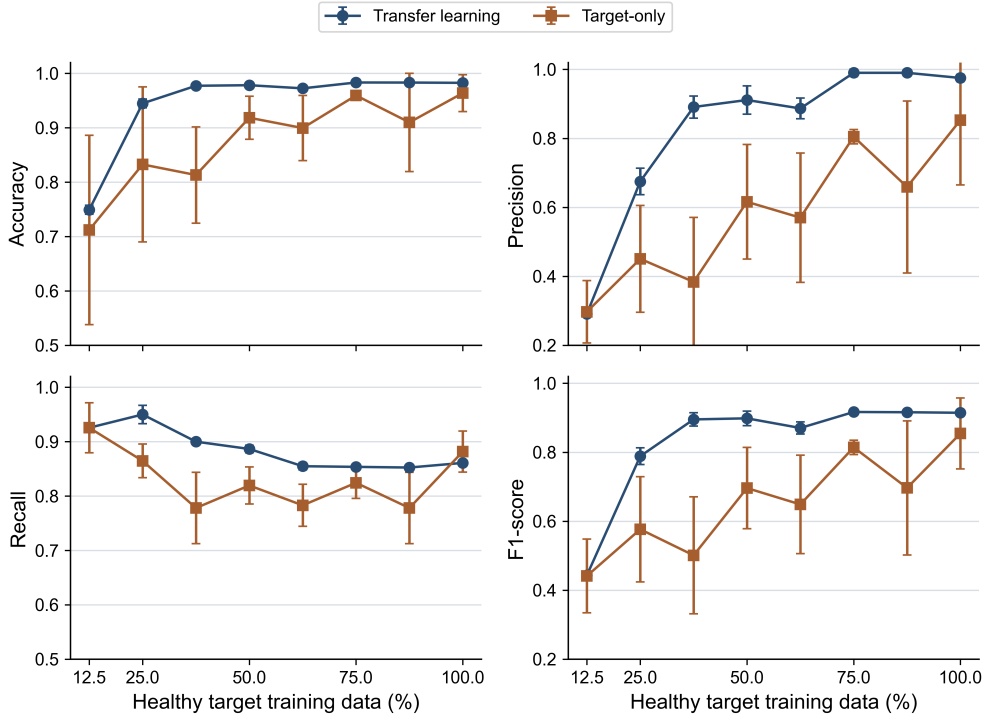


Figure 15: Fault detection performance on the fault-injected target test set $\mathcal{D}_{tgt}^{test-fault}$ under different proportions of healthy target training data. Curves show the mean and error bars show one sample standard deviation over five independent seeds.

6.2.4. Factorial ablation of transfer learning and PROF

Transfer learning (TL) and PROF are independently switched on and off in a controlled 2×2 factorial experiment on the same target-domain fault test set $\mathcal{D}_{\text{tgt}}^{\text{test-fault}}$. All configurations use the same DoE-acquired data, 100% of the designated healthy target-domain training split $\mathcal{D}_{\text{tgt}}^{\text{train}}$, a 30-step input, a 10-step output, the fixed source-calibrated threshold $\tau_{\text{res}} = 1.8$, and identical test samples. “No TL” is the target-only LSTM trained from scratch; “TL” is the source-initialized LSTM with layer 0 frozen. Results are mean $\pm$ sample SD over seeds 42–46.

Table 9

Controlled ablation of transfer learning and PROF on the target-domain fault test set.

Configuration	Accuracy	Precision	Recall	F1-score
No TL + No PROF	0.9469 ± 0.0057	0.7640 ± 0.0341	0.7354 ± 0.0196	0.7493 ± 0.0245
No TL + PROF	0.9638 ± 0.0339	0.8531 ± 0.1882	0.8817 ± 0.0377	0.8546 ± 0.1028
TL + No PROF	0.9532 ± 0.0009	0.8240 ± 0.0052	0.7195 ± 0.0043	0.7682 ± 0.0042
TL + PROF	0.9826 ± 0.0010	0.9752 ± 0.0101	0.8610 ± 0.0027	0.9145 ± 0.0045

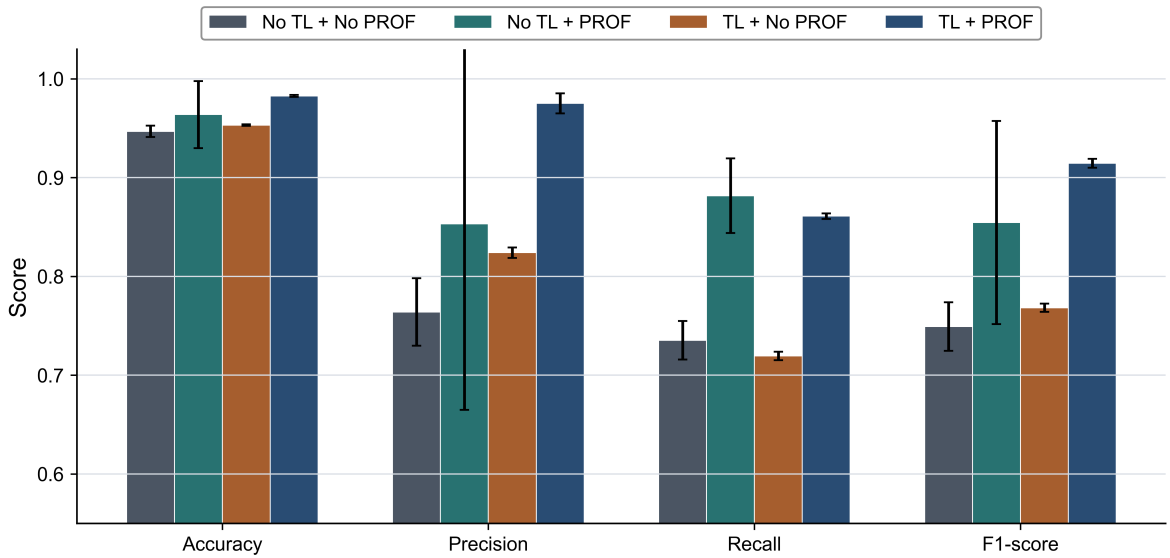


Figure 16: Fault-detection performance in the controlled TL $\times$ PROF ablation. Bars and error bars denote the five-seed mean and sample standard deviation.

The baseline without either component obtains an F1-score of 0.7493 ± 0.0245 . PROF alone raises it to 0.8546 ± 0.1028 , whereas TL alone raises it to 0.7682 ± 0.0042 . Their combination gives the best overall accuracy (0.9826 ± 0.0010), precision (0.9752 ± 0.0101), and F1-score (0.9145 ± 0.0045). Averaging over the paired factorial levels, the main F1-score effect is $+0.1258$ for PROF and $+0.0395$ for TL. PROF therefore provides the larger direct sensitivity gain, while TL reduces nominal cross-domain bias and markedly stabilizes the complete configuration: with PROF enabled, the F1 standard deviation falls from 0.1028 without TL to 0.0045 with TL.

6.3. Deployment-oriented computational efficiency

In this section, we report the training and inference latency of the different models used. Timing was measured on an NVIDIA RTX 1000 Ada Generation Laptop GPU. Inference measurements followed 100 warm-up iterations with explicit GPU synchronization; batch-1 latency used 500 repetitions and batch-64 latency used 200. Model-only timing isolates the forward pass, whereas end-to-end timing includes input transfer and output retrieval. Table 10 reports mean, median, and 95th-percentile latency. The selected LSTM requires a median 0.357 ms per end-to-end batch-1 query and 0.594 ms for batch 64. Its mean batch-1 end-to-end latency of 0.422 ms corresponds to approximately 2368 samples/s under this benchmark.

Table 10

LSTM inference latency and transfer-learning fine-tuning time.

Metric	Condition	Mean	Median	P95 or SD
Inference	Batch 1, model only	0.414 ms	0.460 ms	P95: 0.587 ms
Inference	Batch 1, end to end	0.422 ms	0.357 ms	P95: 0.654 ms
Inference	Batch 64, end to end	0.624 ms	0.594 ms	P95: 0.767 ms
Fine-tuning	100% target training split, five seeds	53.7 s	–	SD: 4.3 s

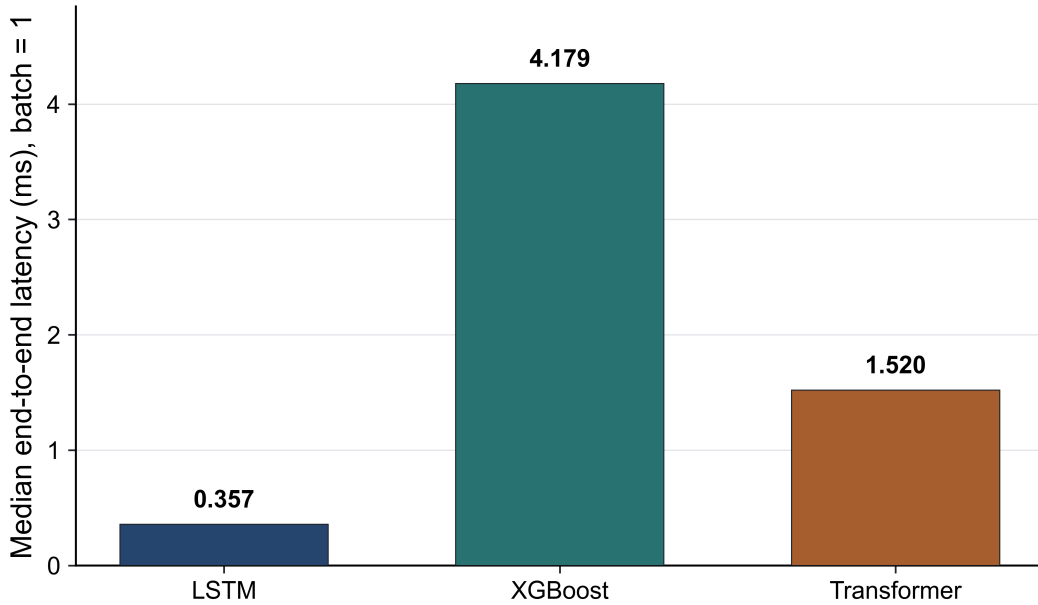
**Figure 17:** Median end-to-end batch-1 inference latency during source-model screening.

Figure 17 compares the same source-screening implementations: LSTM is fastest at 0.357 ms, compared with 1.520 ms for Transformer and 4.179 ms for XGBoost. Transfer-learning time is reported only for LSTM because Section 6.1 selects the backbone before adaptation. Using 100% of the designated verified healthy target-domain training split, freezing layer 0, batch size 64, learning rate 10^{-4} , weight decay 10^{-4} , at most 500 epochs, and patience 10, fine-tuning takes 53.7 ± 4.3 s over seeds 42–46 and stops after 77.2 epochs on average. Online inference is therefore comfortably sub-millisecond on the tested hardware, whereas model adaptation is positioned as an occasional low-frequency maintenance operation outside the real-time detection loop.

7. Conclusions

This study develops an integrated framework for data-driven digital-twin modeling and fault detection. Its main contribution is the integration of three building blocks: (i) a DoE-based data-generation procedure that uses Halton sampling to cover a bounded task-configuration envelope; (ii) a transfer-learning mechanism that adapts the digital twin to system evolution and cross-domain distribution shifts using verified healthy target-domain data; and (iii) a prediction-based recursive observation feedback (PROF) mechanism that prevents detected abnormal observations from contaminating subsequent rolling input windows. Together, these components support reliable nominal modeling and residual-based detection under model drift and fault-induced input contamination.

The framework is validated on two physical robotic arms. The archived LSTM digital twin accurately reconstructs nominal motor-temperature behavior, achieving $R^2 = 0.8591$ and $\text{MAPE} = 0.755\%$ on held-out source-domain test trajectories; it is retained because it offers competitive accuracy, the smallest model footprint, the lowest inference latency, and direct compatibility with parameter transfer. In the controlled residual-alignment experiment reported in

Figs. 12 and 13, direct deployment of this source LSTM on the target test set gives $MSE = 2.8525$ and $R^2 = -0.6200$, whereas adaptation of the same checkpoint reduces MSE to 0.2170 and raises R^2 to 0.8768; the target-to-source Wasserstein residual distance correspondingly decreases from 1.588 to 0.157. This supports reuse of the source-calibrated threshold after healthy target-domain adaptation under matched sensing, preprocessing, and operating conditions, without requiring target-domain fault labels. A controlled 2×2 ablation shows that PROF provides the larger direct F1-score improvement (+0.1258), while transfer learning contributes a smaller but stabilizing effect (F1-score +0.0395, standard deviation reduced from 0.1028 to 0.0045 when PROF is enabled); the complete TL+PROF configuration achieves the best target-domain F1-score of 0.9145. Timing experiments further confirm sub-millisecond online inference (0.357 ms median batch-1 latency) and target-domain fine-tuning in under one minute.

This study has several limitations that point to future work. First, the residual threshold is calibrated on labeled fault-injected source-domain validation data; although no target-domain fault labels are required, the current procedure is not fully independent of labeled fault data, and a normal-data-only threshold based on an upper quantile or statistical confidence bound of healthy residuals is a natural extension. Second, the injected thermal faults follow a first-order lumped-parameter thermal model and represent progressive thermal anomalies rather than the full electromechanical signatures of real motor failures; validation on naturally occurring or safely reproduced physical faults would provide stronger evidence. Third, the verify-then-update policy assumes that gradual healthy drift requiring model adaptation can be distinguished from fault signals that must be excluded; fully automatic discrimination between the two is not addressed and remains an important direction for future work.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used ChatGPT and GLM-5.2 to refine the language of the completed manuscript. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] G. Qiao, B. A. Weiss, Quick health assessment for industrial robot health degradation and the supporting advanced sensing development, *Journal of Manufacturing Systems* 48 (2018) 51–59. URL: <https://www.sciencedirect.com/science/article/pii/S0278612518300438>. doi:<https://doi.org/10.1016/j.jmsy.2018.04.004>, special Issue on Smart Manufacturing.
- [2] Y. Lei, H. Liu, N. Li, J. Cao, Y. Qiao, H. Wang, Condition monitoring and fault diagnosis of industrial robots: A review, *Science China Technological Sciences* 68 (2025) 1110301.
- [3] A. H. Sabry, U. A. B. Ungku Amirulddin, A review on fault detection and diagnosis of industrial robots and multi-axis machines, *Results in Engineering* 23 (2024) 102397. URL: <https://www.sciencedirect.com/science/article/pii/S2590123024006522>. doi:<https://doi.org/10.1016/j.rineng.2024.102397>.
- [4] K. F. Ávila Okada, A. Silva de Moraes, L. C. Oliveira-Lopes, L. Ribeiro, A survey on fault detection and diagnosis methods, in: 2021 14th IEEE International Conference on Industry Applications (INDUSCON), 2021, pp. 1422–1429. doi:10.1109/INDUSCON51756.2021.9529495.
- [5] H. H. Hosamo, H. K. Nielsen, A. N. Alnmr, P. R. Svennevig, K. Svidt, A review of the digital twin technology for fault detection in buildings, *Frontiers in Built Environment* 8 (2022) 1013196.
- [6] H. Darvishi, D. Ciuonzo, E. R. Eide, P. S. Rossi, Sensor-fault detection, isolation and accommodation for digital twins via modular data-driven architecture, *IEEE Sensors Journal* 21 (2020) 4827–4838.
- [7] C. Hu, Z. Zhang, C. Li, M. Leng, Z. Wang, X. Wan, C. Chen, A state of the art in digital twin for intelligent fault diagnosis, *Advanced Engineering Informatics* 63 (2025) 102963. URL: <https://www.sciencedirect.com/science/article/pii/S1474034624006141>. doi:<https://doi.org/10.1016/j.aei.2024.102963>.
- [8] Y. You, C. Chen, F. Hu, Y. Liu, Z. Ji, Advances of digital twins for predictive maintenance, *Procedia Computer Science* 200 (2022) 1471–1480. URL: <https://www.sciencedirect.com/science/article/pii/S187705092200357X>. doi:<https://doi.org/10.1016/j.procs.2022.01.348>, 3rd International Conference on Industry 4.0 and Smart Manufacturing.
- [9] S. Y. Teng, M. Touš, W. D. Leong, B. S. How, H. L. Lam, V. Máša, Recent advances on industrial data-driven energy savings: Digital twins and infrastructures, *Renewable and Sustainable Energy Reviews* 135 (2021) 110208.
- [10] J. Friederich, D. P. Francis, S. Lazarova-Molnar, N. Mohamed, A framework for data-driven digital twins of smart manufacturing systems, *Computers in Industry* 136 (2022) 103586.
- [11] A. P. Renold, N. S. Kathayat, Comprehensive review of machine learning, deep learning, and digital twin data-driven approaches in battery health prediction of electric vehicles, *IEEE Access* 12 (2024) 43984–43999. doi:10.1109/ACCESS.2024.3380452.

- [12] L. Hui, M. Wang, L. Zhang, L. Lu, Y. Cui, Digital twin for networking: A data-driven performance modeling perspective, *IEEE Network* 37 (2022) 202–209.
- [13] G. Michau, Y. Hu, T. Palmé, O. Fink, Feature learning for fault detection in high-dimensional condition monitoring signals, *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability* 234 (2019) 104–115. URL: <http://dx.doi.org/10.1177/1748006X19868335>. doi:10.1177/1748006X19868335.
- [14] D. Zhong, Z. Xia, Y. Zhu, J. Duan, Overview of predictive maintenance based on digital twin technology, *Heliyon* 9 (2023) e14534. URL: <https://www.sciencedirect.com/science/article/pii/S2405844023017413>. doi:<https://doi.org/10.1016/j.heliyon.2023.e14534>.
- [15] D. Leite, E. Andrade, D. Rativa, A. M. Maciel, Fault detection and diagnosis in industry 4.0: a review on challenges and opportunities, *Sensors (Basel, Switzerland)* 25 (2024) 60.
- [16] R. Isermann, P. Ballé, Trends in the application of model-based fault detection and diagnosis of technical processes, *Control Engineering Practice* 5 (1997) 709–719. URL: <https://www.sciencedirect.com/science/article/pii/S0967066197000531>. doi:[https://doi.org/10.1016/S0967-0661\(97\)00053-1](https://doi.org/10.1016/S0967-0661(97)00053-1).
- [17] J. Marzat, H. Piet-Lahanier, F. Damongeot, E. Walter, Model-based fault diagnosis for aerospace systems: a survey, *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of aerospace engineering* 226 (2012) 1329–1360.
- [18] Z. Zemali, L. Cherroun, N. Hadroug, M. Nadour, A. Hafaifa, Fault diagnosis-based observers using kalman filters and luenberger estimators: Application to the pitch system fault actuators, *Diagnostyka* 24 (2023) 1–13. URL: <https://doi.org/10.29354/diag/161307>. doi:10.29354/diag/161307.
- [19] H. Ballesteros-Moncada, E. J. Herrera-López, J. Anzurez-Marín, Fuzzy model-based observers for fault detection in cstr, *ISA Transactions* 59 (2015) 325–333. URL: <https://www.sciencedirect.com/science/article/pii/S0019057815002402>. doi:<https://doi.org/10.1016/j.isatra.2015.10.006>.
- [20] D. Jung, Engine fault diagnosis combining model-based residuals and data-driven classifiers, *IFAC-PapersOnLine* 52 (2019) 285–290. URL: <https://www.sciencedirect.com/science/article/pii/S2405896319306664>. doi:<https://doi.org/10.1016/j.ifacol.2019.09.046>, 9th IFAC Symposium on Advances in Automotive Control AAC 2019.
- [21] D. Aiordăchioaie, A comparative analysis of fault detection and process diagnosis methods based on a signal processing paradigm, *Discover Applied Sciences* 7 (2024) 10.
- [22] H. Wu, C. Wang, Survey of gear fault feature extraction methods based on signal processing, in: *International Conference in Communications, Signal Processing, and Systems*, Springer, 2019, pp. 1494–1504.
- [23] Z. Jiang, K. Zhang, L. Xiang, G. Yu, Y. Xu, A time-frequency spectral amplitude modulation method and its applications in rolling bearing fault diagnosis, *Mechanical Systems and Signal Processing* 185 (2023) 109832. URL: <https://www.sciencedirect.com/science/article/pii/S0888327022009001>. doi:<https://doi.org/10.1016/j.ymssp.2022.109832>.
- [24] P. Zhou, S. Chen, Q. He, D. Wang, Z. Peng, Rotating machinery fault-induced vibration signal modulation effects: A review with mechanisms, extraction methods and applications for diagnosis, *Mechanical Systems and Signal Processing* 200 (2023) 110489. URL: <https://www.sciencedirect.com/science/article/pii/S0888327023003977>. doi:<https://doi.org/10.1016/j.ymssp.2023.110489>.
- [25] A. R. Sahu, S. K. Palei, A. Mishra, Data-driven fault diagnosis approaches for industrial equipment: A review, *Expert Systems* 41 (2024) e13360.
- [26] X. Dai, Z. Gao, From model, signal to knowledge: A data-driven perspective of fault detection and diagnosis, *IEEE Transactions on Industrial Informatics* 9 (2013) 2226–2238.
- [27] Z. Chen, Z. O'Neill, J. Wen, O. Pradhan, T. Yang, X. Lu, G. Lin, S. Miyata, S. Lee, C. Shen, et al., A review of data-driven fault detection and diagnostics for building hvac systems, *Applied Energy* 339 (2023) 121030.
- [28] F. Calabrese, A. Regattieri, M. Bortolini, F. G. Galizia, Data-driven fault detection and diagnosis: Challenges and opportunities in real-world scenarios, *Applied Sciences* 12 (2022) 9212.
- [29] R. M. Souza, E. G. Nascimento, U. A. Miranda, W. J. Silva, H. A. Lepikson, Deep learning for diagnosis and classification of faults in industrial rotating machinery, *Computers & Industrial Engineering* 153 (2021) 107060. URL: <https://www.sciencedirect.com/science/article/pii/S0360835220307300>. doi:<https://doi.org/10.1016/j.cie.2020.107060>.
- [30] D. Neupane, M. R. Bouadjenek, R. Dazeley, S. Aryal, Data-driven machinery fault diagnosis: A comprehensive review, *Neurocomputing* (2025) 129588.
- [31] J. M. Ramírez-Sanz, J.-A. Maestro-Prieto, Á. Arnaiz-González, A. Bustillo, Semi-supervised learning for industrial fault detection and diagnosis: A systemic review, *ISA transactions* 143 (2023) 255–270.
- [32] A. Saeed, M. A. Khan, U. Akram, W. J. Obidallah, S. Jawed, A. Ahmad, Deep learning based approaches for intelligent industrial machinery health management and fault diagnosis in resource-constrained environments, *Scientific Reports* 15 (2025) 1114.
- [33] M. Javaid, A. Haleem, R. Suman, Digital twin applications toward industry 4.0: A review, *Cognitive Robotics* 3 (2023) 71–92.
- [34] P. V. Lopes, L. Silveira, R. D. Guimaraes Aquino, C. H. Ribeiro, A. Skoogh, F. A. N. Verri, Synthetic data generation for digital twins: enabling production systems analysis in the absence of data, *International Journal of Computer Integrated Manufacturing* 37 (2024) 1252–1269.
- [35] R. Van Dinter, B. Tekinerdogan, C. Catal, Predictive maintenance using digital twins: A systematic literature review, *Information and Software Technology* 151 (2022) 107008.
- [36] J. Hu, H. Xiao, Z. Ye, N. Luo, M. Zhou, Research and prospects of digital twin-based fault diagnosis of electric machines, *Sensors* 25 (2025) 2625.
- [37] J. Xia, R. Huang, Z. Chen, G. He, W. Li, A novel digital twin-driven approach based on physical-virtual data fusion for gearbox fault diagnosis, *Reliability Engineering & System Safety* 240 (2023) 109542.
- [38] J. Xia, Z. Chen, J. Chen, G. He, R. Huang, W. Li, A digital twin-driven approach for partial domain fault diagnosis of rotating machinery, *Engineering Applications of Artificial Intelligence* 131 (2024) 107848.

- [39] T. Kreuzer, P. Papapetrou, J. Zdravkovic, Artificial intelligence in digital twins—a systematic literature review, *Data & Knowledge Engineering* 151 (2024) 102304.
- [40] L. Nele, G. Mattera, E. W. Yap, M. Vozza, S. Vespoli, Towards the application of machine learning in digital twin technology: a multi-scale review, *Discover Applied Sciences* 6 (2024) 502.
- [41] T. Khan, U. Khan, A. Khan, C. Mollan, I. Morkvenaite-Vilkonciene, V. Pandey, Data-driven digital twin framework for predictive maintenance of smart manufacturing systems, *Machines* 13 (2025) 481.
- [42] S. Cheruku, S. Balaji, A. Delgado, V. R. Krishnamurthy, Data-driven digital twins for real-time machine monitoring: A case study on a rotating machine, *Journal of Computing and Information Science in Engineering* 25 (2025) 031005.
- [43] R. Rodriguez-Aguilar, J.-A. Marmolejo-Saucedo, U. Köse, Development of a digital twin driven by a deep learning model for fault diagnosis of electro-hydrostatic actuators, *Mathematics* 12 (2024) 3124.
- [44] H. Chi, M. Mascagni, T. Warnock, On the optimal halton sequence, *Mathematics and computers in simulation* 70 (2005) 9–21.
- [45] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, Y. C. Eldar, KalmanNet: Neural network aided Kalman filtering for partially known dynamics, *IEEE Transactions on Signal Processing* 70 (2022) 1532–1547. doi:10.1109/TSP.2022.3158588.
- [46] B. Lusch, J. N. Kutz, S. L. Brunton, Deep learning for universal linear embeddings of nonlinear dynamics, *Nature Communications* 9 (2018) 4950. doi:10.1038/s41467-018-07210-0.
- [47] A. Zarchini, D. Dowling, F. Golpayegani, Print defect detection during laser powder bed fusion by one-dimensional convolutional autoencoder, *INTERNATIONAL JOURNAL OF ADVANCED MANUFACTURING TECHNOLOGY* (2026). doi:10.1007/s00170-026-18644-y.
- [48] J. Li, D. Ding, F. Tsung, Directional pca for fast detection and accurate diagnosis: A unified framework, *IEEE TRANSACTIONS ON CYBERNETICS* 52 (2022) 11362–11372. doi:10.1109/TCYB.2021.3070590.
- [49] Y. Li, J. Xie, L. Chen, Y. Zeng, X. Chen, T. Pan, Time-frequency reconstruction discrepancy-guided domain adaptation for open-set fault diagnosis of train bogie bearing, *ISA TRANSACTIONS* 175 (2026) 184–200. doi:10.1016/j.isatra.2026.05.022.
- [50] S. Khalesidoost, J. Faiz, E. Mazaheri-Tehrani, An overview of thermal modelling techniques for permanent magnet machines, *IET Science, Measurement & Technology* 16 (2022) 219–241.
- [51] C. Sciascera, P. Giangrande, L. Papini, C. Gerada, M. Galea, Analytical thermal model for fast stator winding temperature prediction, *IEEE Transactions on Industrial Electronics* 64 (2017) 6116–6126.
- [52] A. Boglietti, A. Cavagnino, D. Staton, M. Shanel, M. Mueller, C. Mejuto, Evolution and modern approaches for thermal analysis of electrical machines, *IEEE Transactions on industrial electronics* 56 (2009) 871–882.
- [53] S. Ramarathnam, A. K. Mohammed, B. Bilgin, A. Sathyan, H. Dadkhah, A. Emadi, A review of structural and thermal analysis of traction motors, *IEEE Transactions on Transportation Electrification* 1 (2015) 255–265.